

(12) **United States Patent**
McCord et al.

(10) **Patent No.:** **US 12,388,880 B2**
(45) **Date of Patent:** **Aug. 12, 2025**

(54) **SECURITY VALIDATION SYSTEM USING DYNAMIC RELATIONSHIP FINGERPRINTS**

(71) Applicant: **Intelligent Communication Assistant, Inc.**, Boca Raton, FL (US)

(72) Inventors: **Alan W McCord**, Forney, TX (US); **Gustavo Manuel Damil Marin**, Point Roberts, WA (US); **Raymond J. Sheppard**, Spring Hill, FL (US)

(73) Assignee: **Intelligent Communication Assistant, Inc.**, Boca Raton, FL (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **19/066,192**

(22) Filed: **Feb. 28, 2025**

(65) **Prior Publication Data**
US 2025/0202956 A1 Jun. 19, 2025

Related U.S. Application Data

(63) Continuation-in-part of application No. 18/921,443, filed on Oct. 21, 2024, now Pat. No. 12,261,893, which is a continuation of application No. 18/751,905, filed on Jun. 24, 2024, now Pat. No. 12,155,700.

(60) Provisional application No. 63/601,645, filed on Nov. 21, 2023.

(51) **Int. Cl.**
H04L 65/1076 (2022.01)
H04L 51/02 (2022.01)
H04L 65/1096 (2022.01)

(52) **U.S. Cl.**
CPC **H04L 65/1076** (2013.01); **H04L 51/02** (2013.01); **H04L 65/1096** (2013.01)

(58) **Field of Classification Search**
None
See application file for complete search history.

(56) **References Cited**
U.S. PATENT DOCUMENTS

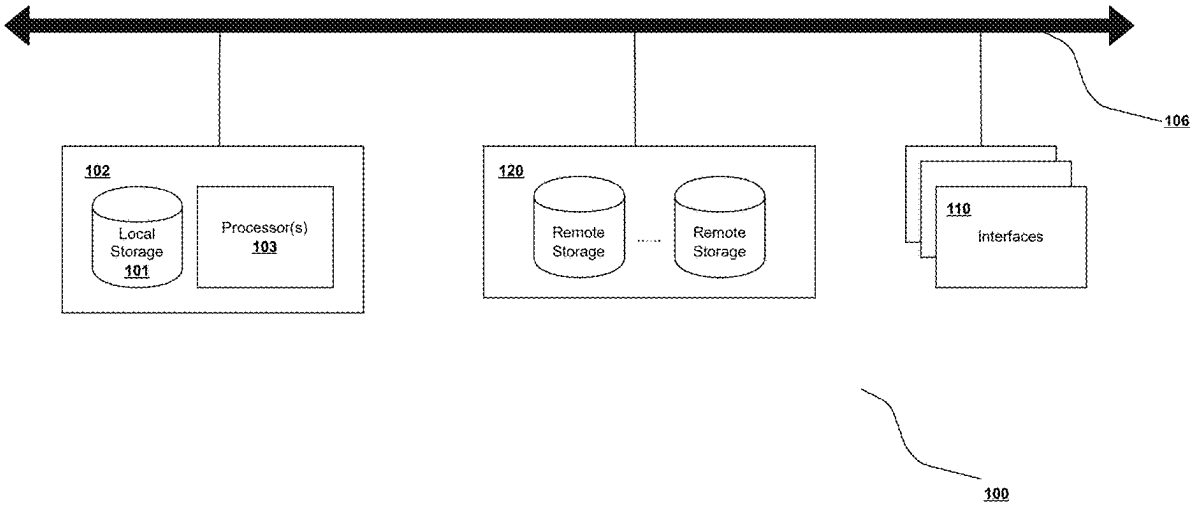
9,955,215 B2 * 4/2018 Davis H04N 21/23
11,893,464 B1 * 2/2024 Everest G06F 21/31
2018/0007199 A1 * 1/2018 Quilici H04M 15/41
2019/0172073 A1 * 6/2019 Wiig G06Q 10/06315
2022/0343095 A1 * 10/2022 Naqvi H04L 63/102

(Continued)

Primary Examiner — Raqiul A Choudhury
(74) *Attorney, Agent, or Firm* — Marin Patents LLC; Gustavo Marin

(57) **ABSTRACT**
A system and method for security validation of incoming communications through relationship fingerprints is disclosed. The system analyzes metadata from incoming communications to identify communication patterns without accessing content. Using these patterns, the system validates relationship context between communicating parties through an interaction graph and relationship fingerprints stored in a database. The relationship fingerprints contain temporal patterns, channel preferences, and behavioral patterns that evolve as relationships develop. The system validates incoming communications by matching current patterns against stored relationship fingerprints. Additionally, the system enables cross-service pattern sharing through aggregated patterns that can be normalized and shared across networks while maintaining privacy. The system continuously updates both relationship fingerprints and aggregated patterns as new communications occur, creating an evolving security framework based on relationship patterns.

18 Claims, 14 Drawing Sheets



(56)

References Cited

U.S. PATENT DOCUMENTS

2024/0107316	A1 *	3/2024	Kim	H04W 12/037
2024/0396998	A1 *	11/2024	Ashfield	H04M 3/4365
2025/0078192	A1 *	3/2025	Gu	G06F 21/32

* cited by examiner

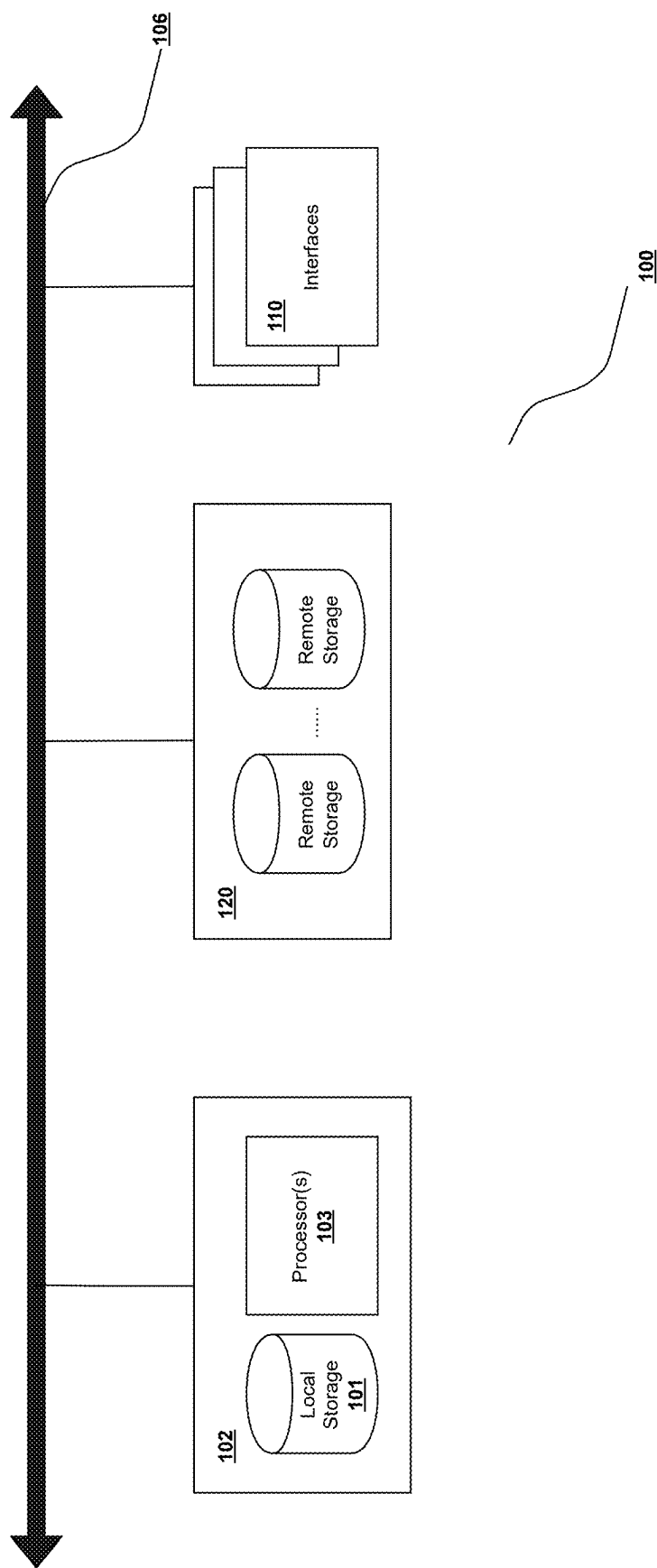


Fig. 1

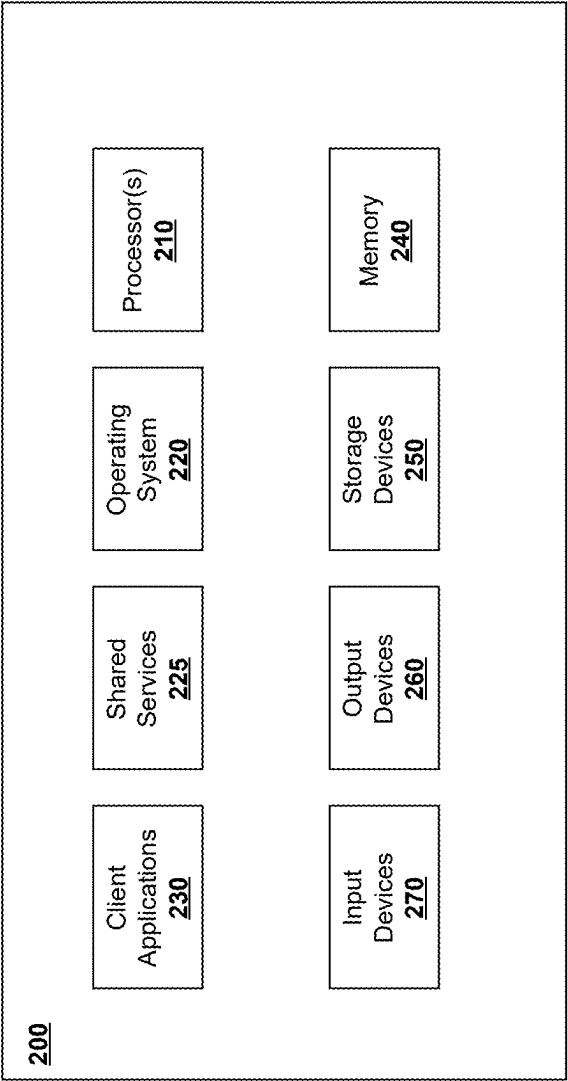


Fig. 2

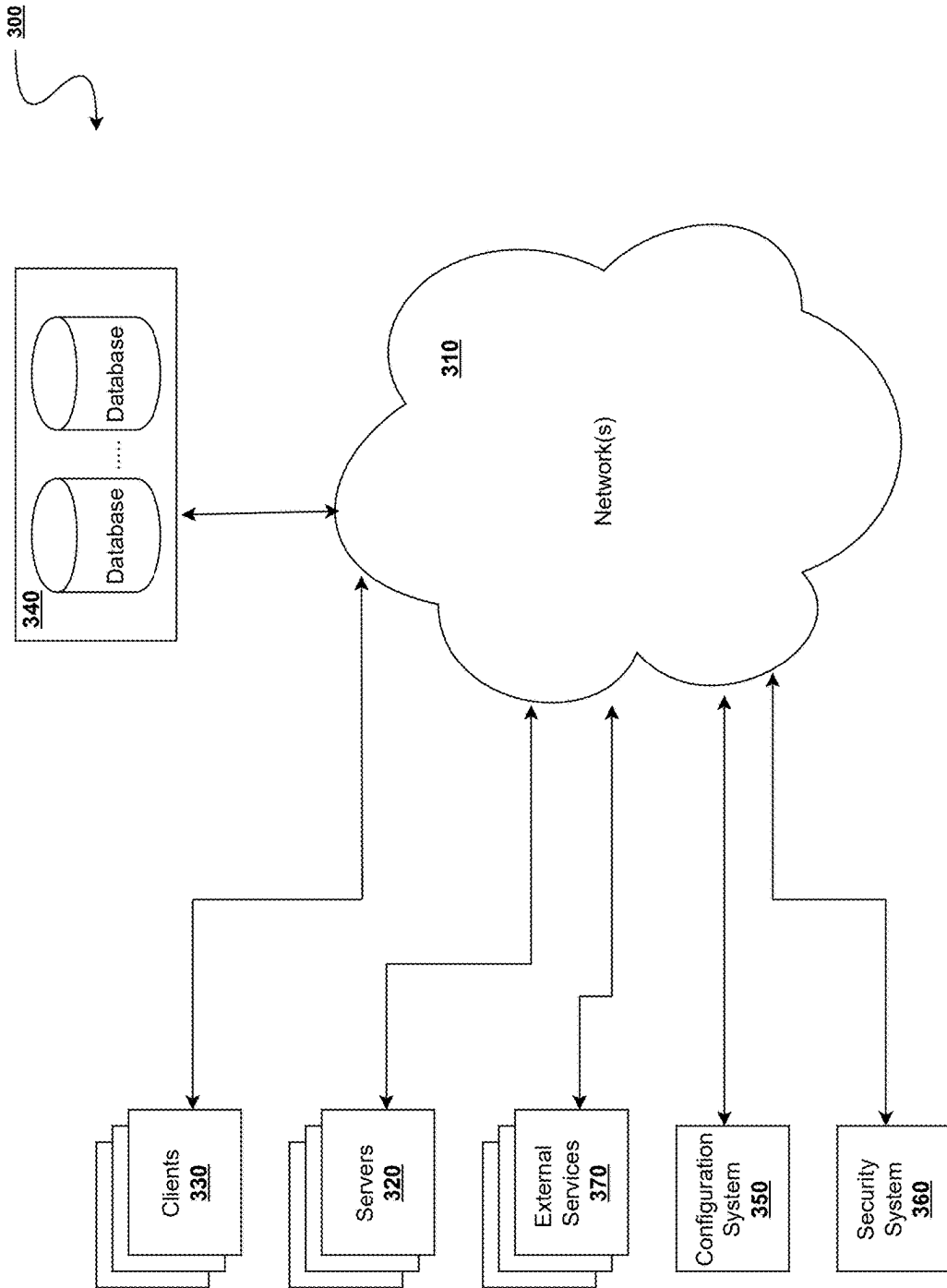


Fig. 3

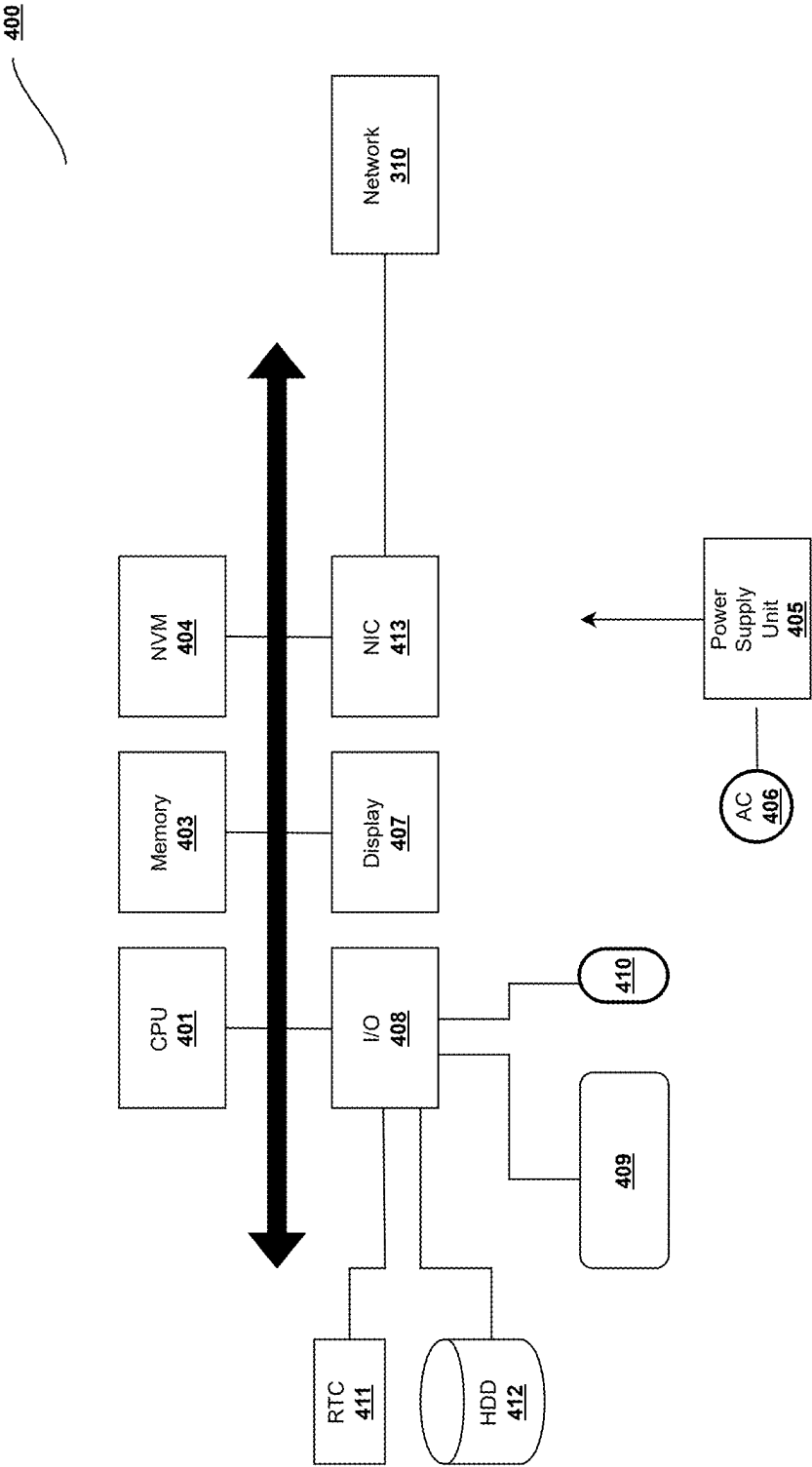


Fig. 4

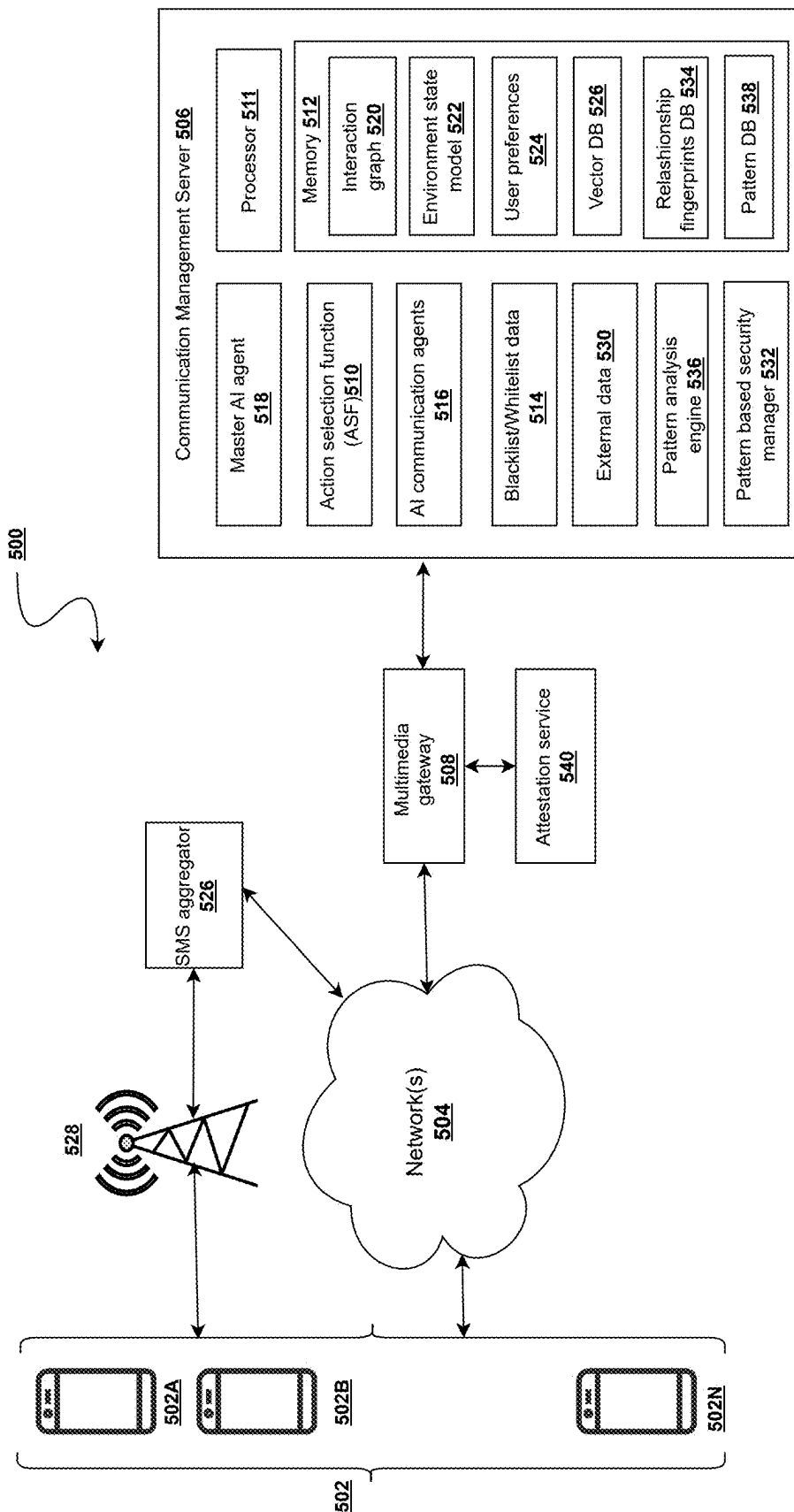


Fig. 5

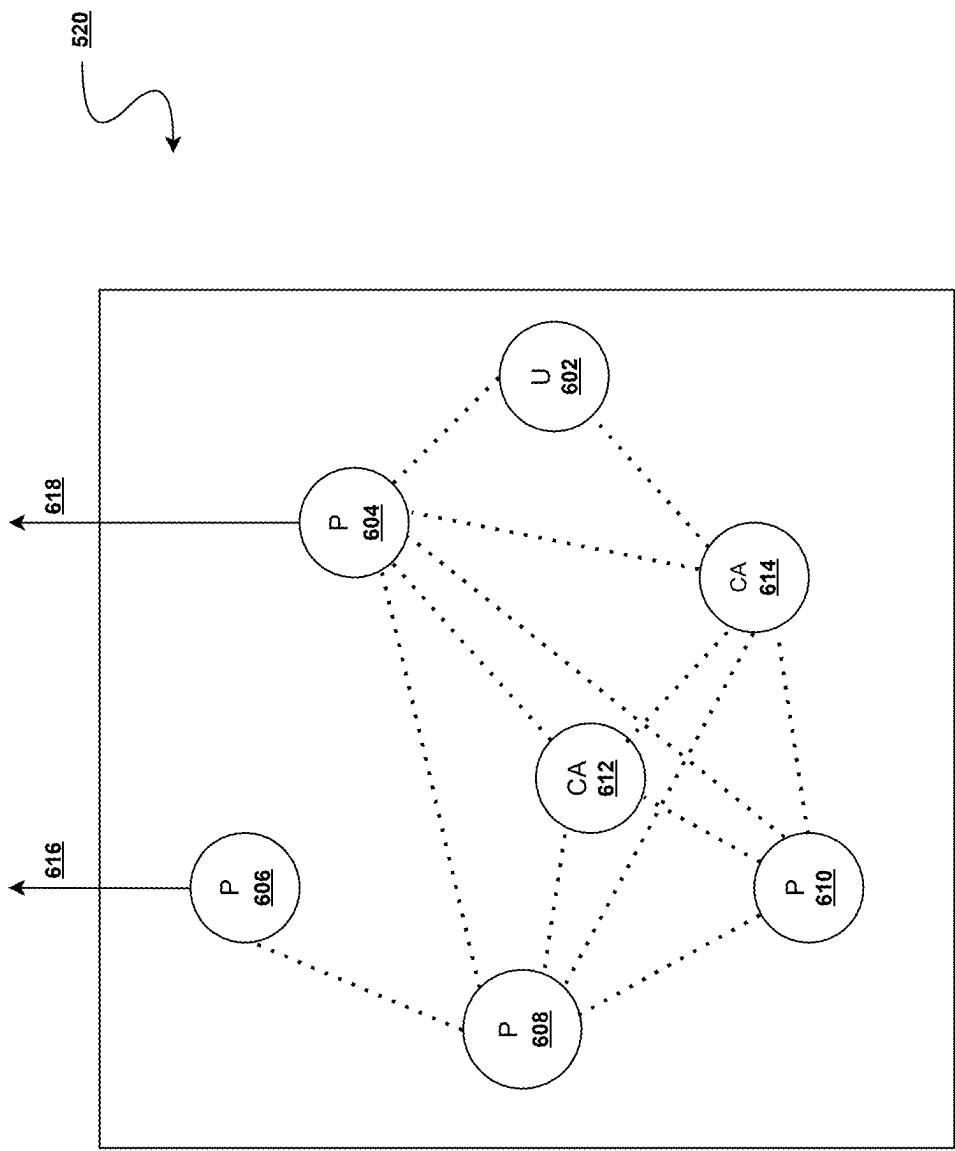


Fig. 6

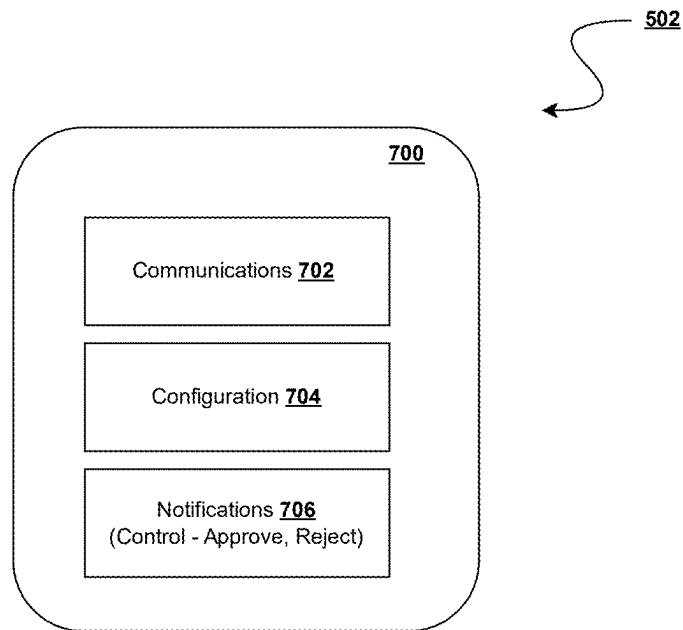


Fig. 7A

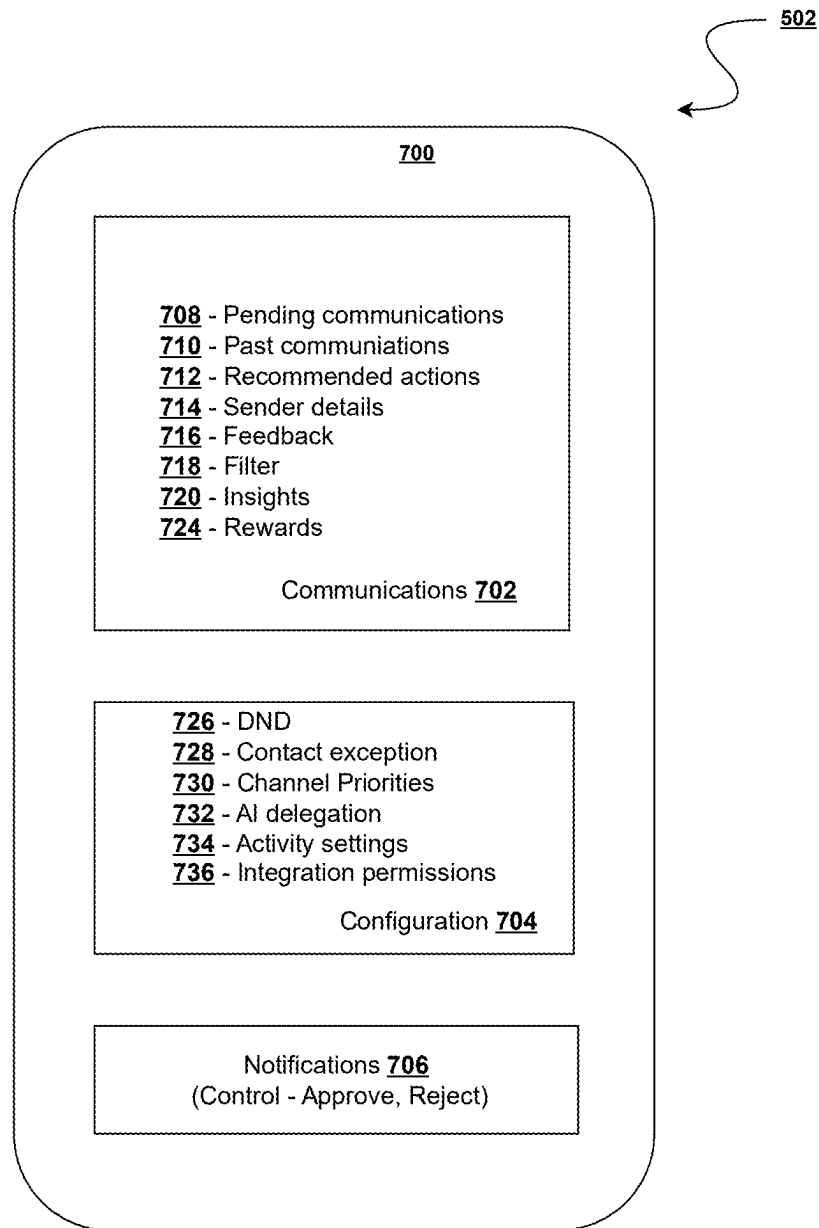


Fig. 7B

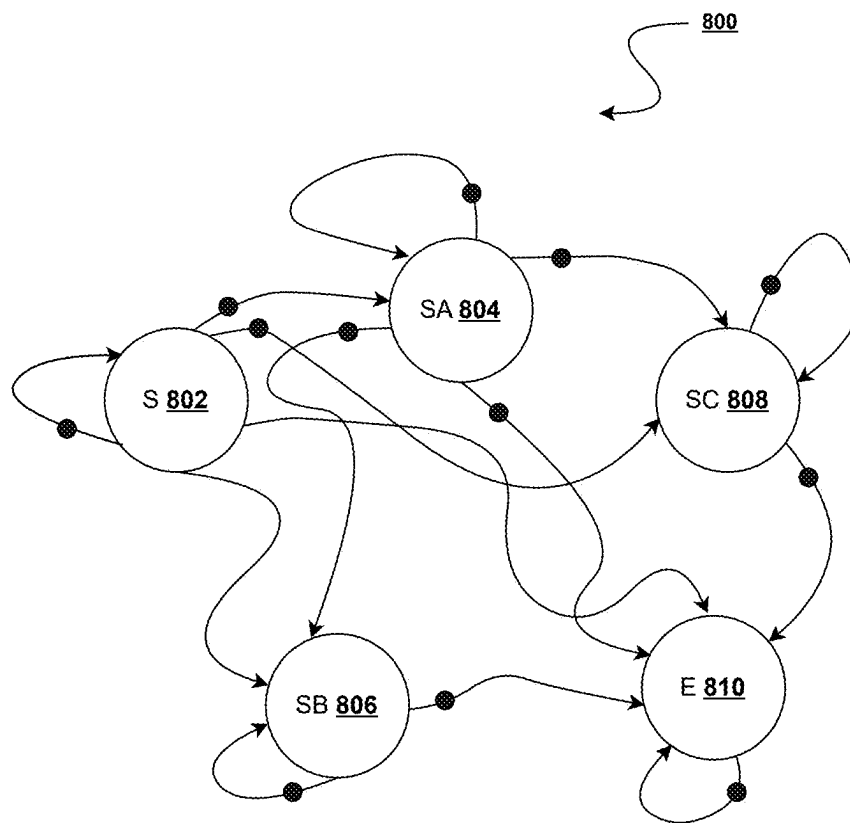


Fig. 8

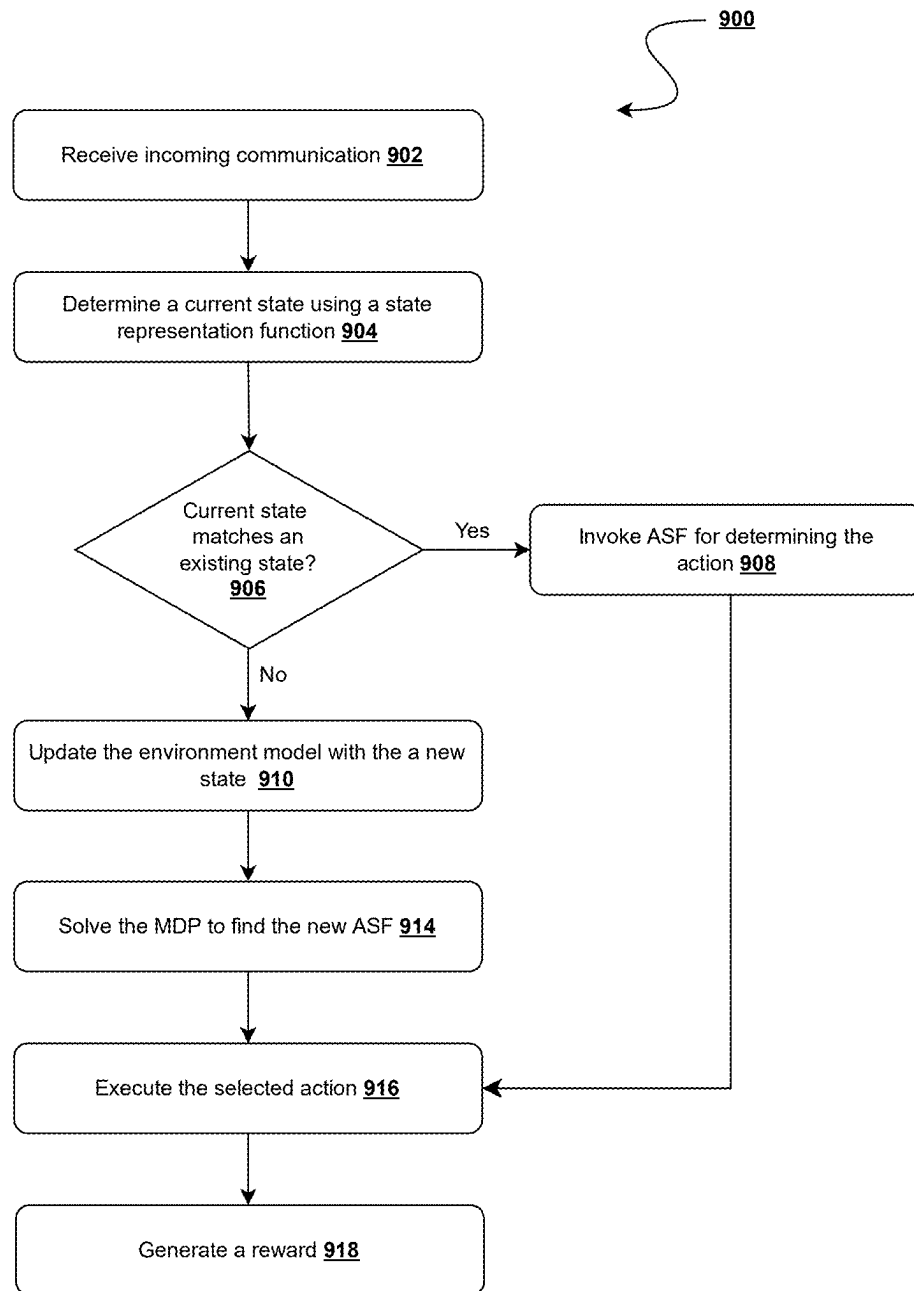


Fig. 9

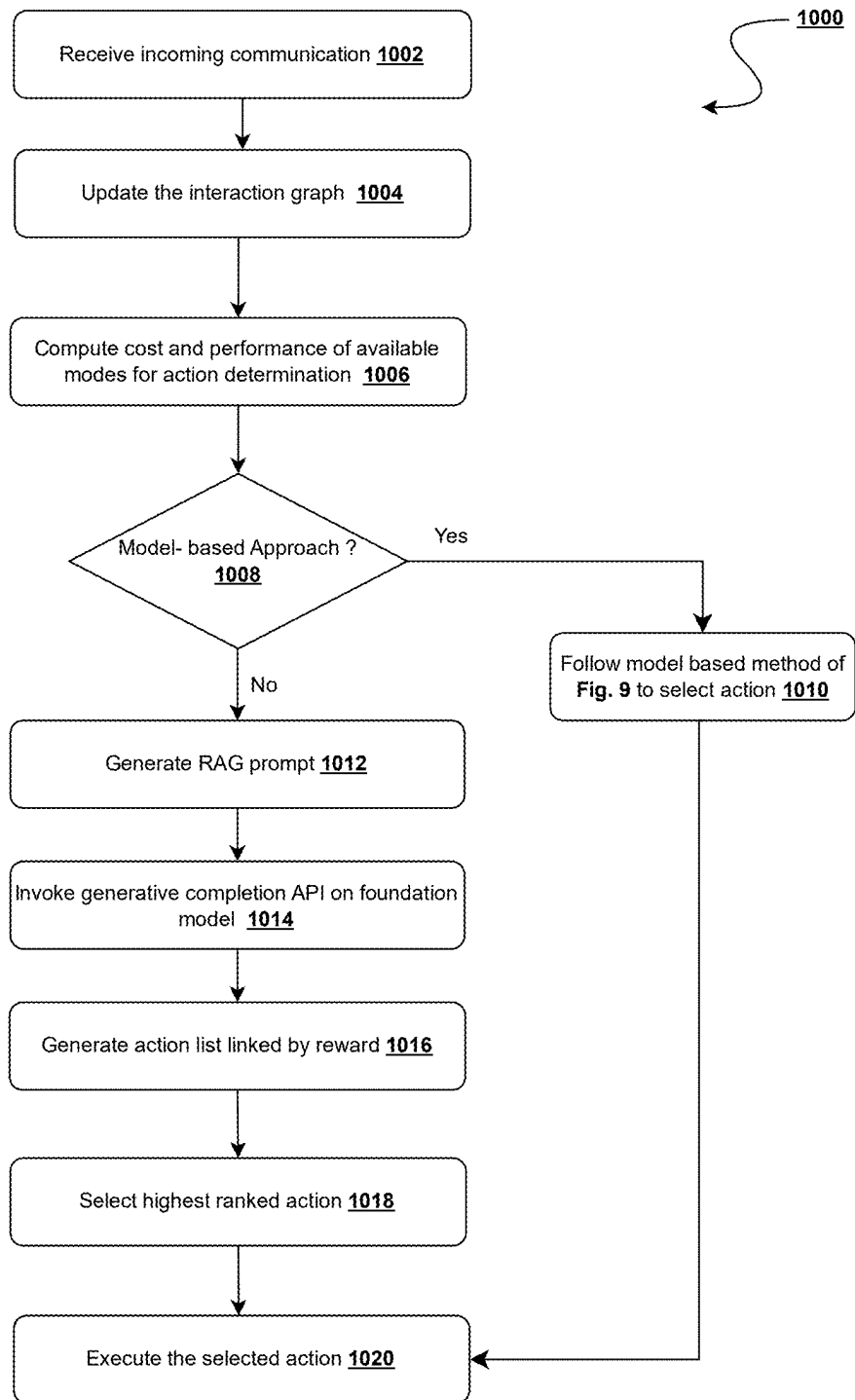


Fig. 10

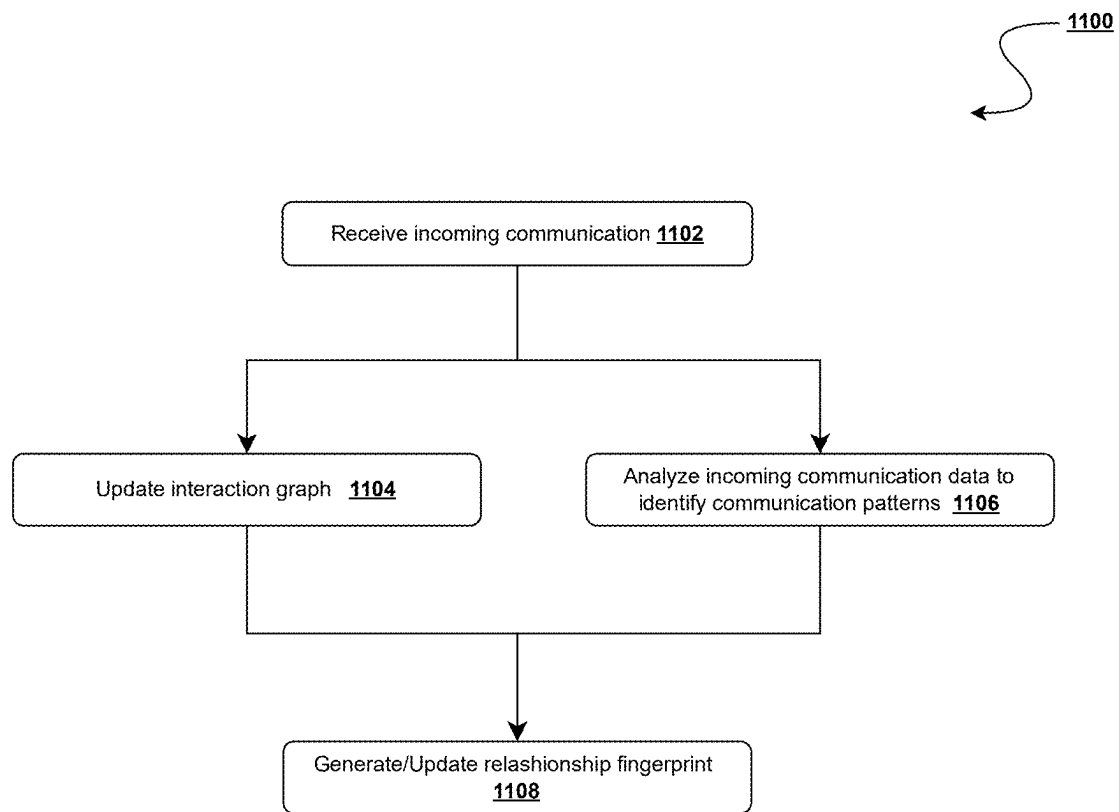


Fig. 11

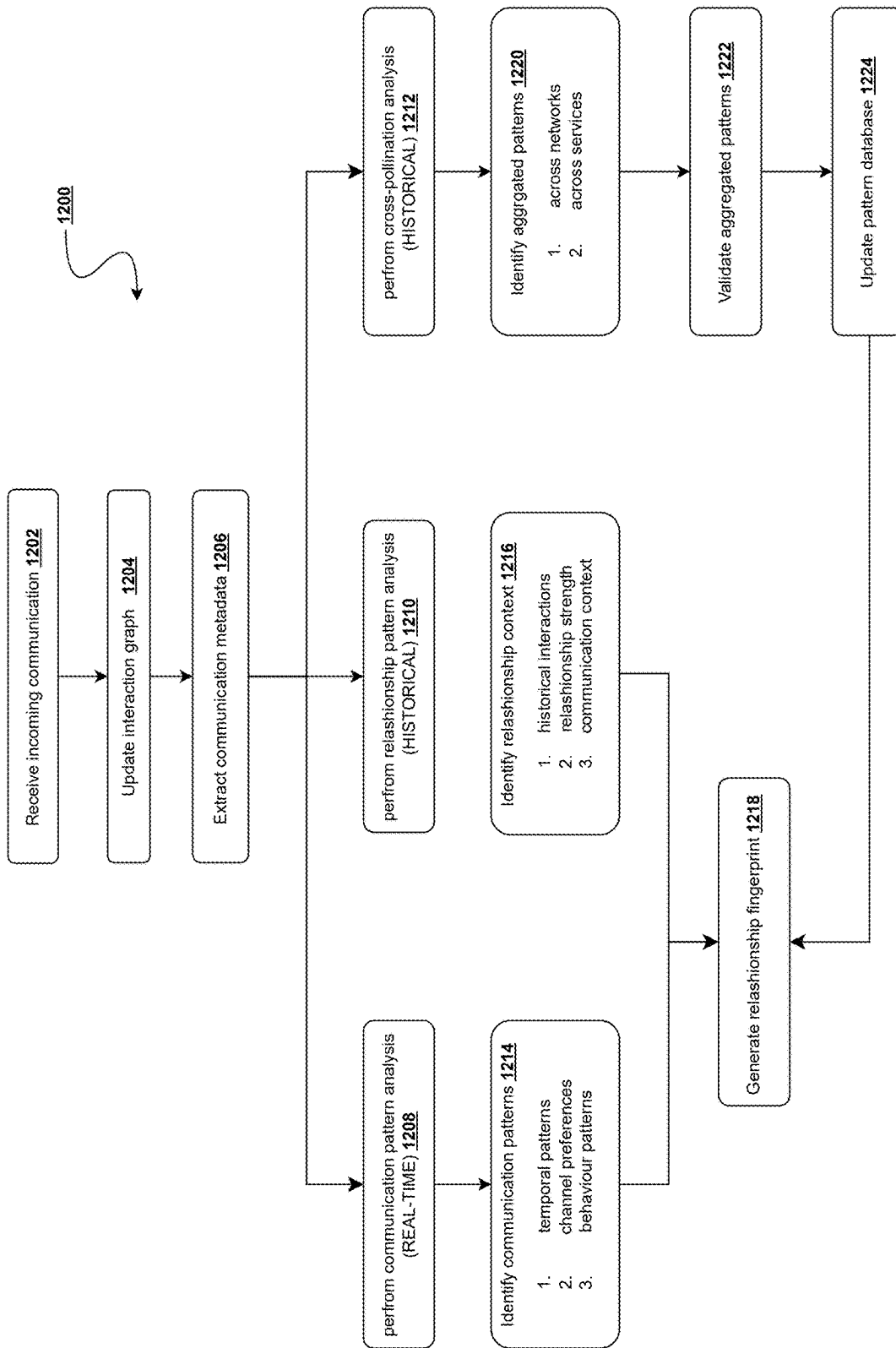


Fig. 12

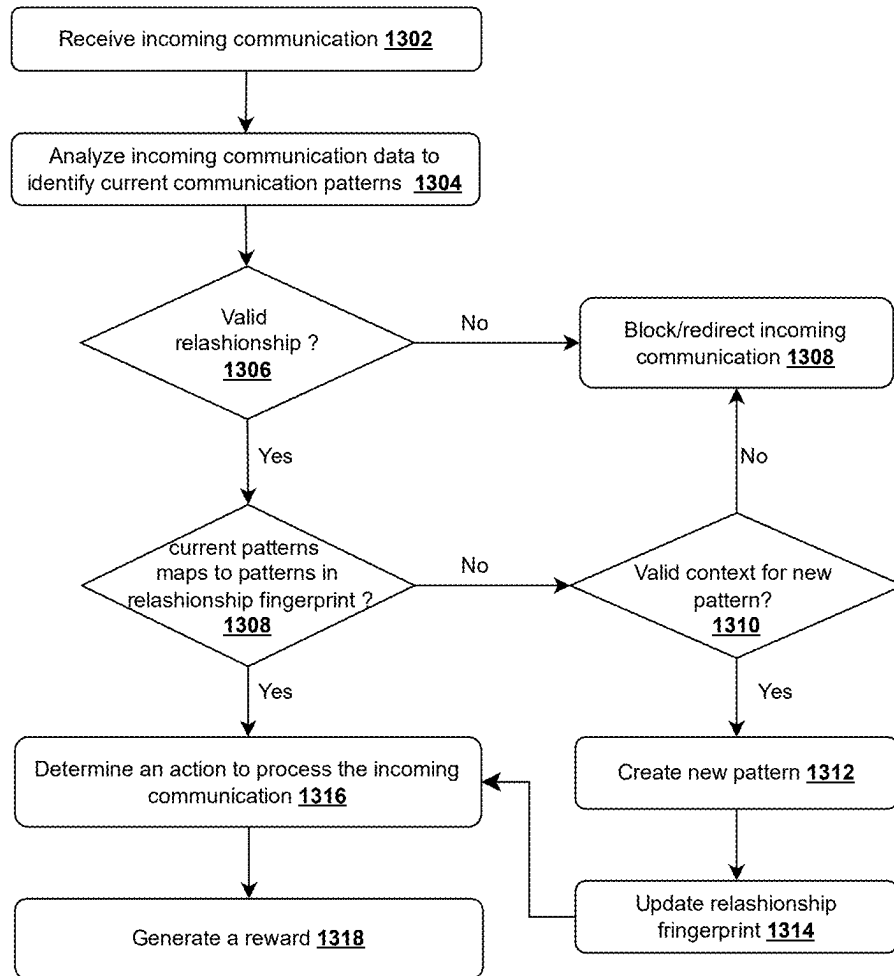


Fig. 13

SECURITY VALIDATION SYSTEM USING DYNAMIC RELATIONSHIP FINGERPRINTS

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of U.S. patent application Ser. No. 18/921,443, titled, “ADAPTIVE COMMUNICATION MANAGEMENT SYSTEM USING MODEL-BASED AND MODEL FREE REINFORCEMENT LEARNING” filed on Oct. 21, 2024 which claims the benefit of, and priority to, U.S. patent application Ser. No. 18/751,905, titled, “SYSTEMS AND METHOD FOR OPTIMIZING PERSONAL COMMUNICATIONS THROUGH OBJECTIVE ALIGNMENT” which claims the benefit of, and priority to, U.S. Provisional Application No. 63/601,645, titled, “SYSTEM AND METHOD FOR OPTIMIZING PERSONAL COMMUNICATIONS THROUGH OBJECTIVE ALIGNMENT” filed on Nov. 21, 2023, the specifications of which are hereby incorporated by reference in its entirety.

BACKGROUND OF THE INVENTION

Field of the Art

The disclosure relates to the field of communications, and more particularly to the field of securing communications through relationship patterns.

Discussion of the State of the Art

Current communication security systems face increasing challenges in the modern artificial intelligence (AI) era. Conventional caller identification systems rely on data packets that may be easily spoofed, making it difficult to verify the true origin of communications. Although protocols like STIR/SHAKEN have been implemented to authenticate voice calls, their effectiveness is limited by their narrow scope and reliance on certificate-based attestation systems, which remain susceptible to manipulation by sophisticated threat actors.

Further, existing solutions that attempt to combat these threats actors through identity verification, content monitoring, blocklists, behavioral analysis, and authentication protocols compromise user privacy by requiring access to sensitive data. While machine learning solutions may be considered as potential solution, they require access to large training datasets, require constant retraining, and collect extensive personal data resulting in compromising privacy for functionality while remaining vulnerable to adversarial AI attacks.

Hence, there is a need for a new approach to communication security that can protect users from evolved AI-powered threats without relying on identity authentication or access to users' content.

SUMMARY OF THE INVENTION

Accordingly, the inventor has conceived and reduced to practice, in a preferred embodiment of the invention, a system for security validation of incoming communications using relationship fingerprints. The system analyzes communication metadata to identify communication patterns without accessing content, validates relationship context between communicating parties, and uses evolving relationship fingerprints communications.

According to a preferred embodiment of the invention, the system includes a multimedia gateway configured to manage real-time media and a communication management server. When an incoming communication is received, a pattern analysis engine analyzes metadata associated with the incoming communication to identify communication patterns such as timing, channel preferences, and behavioral patterns. A pattern-based decision manager validates the relationship context between communicating parties using both an interaction graph and relationship fingerprints.

Relationship fingerprints capture unique characteristics of how parties communicate, including temporal patterns, channel preferences, and behavioral patterns. These fingerprints naturally evolve as relationships develop, making them increasingly precise in distinguishing legitimate communications from unwanted ones.

The system performs cross-pollination analysis to identify common patterns across services and networks. When similar patterns are observed across different services (email, voice, messaging), the system validates these patterns and normalizes them for sharing across networks. This creates a self-strengthening security framework where patterns identified in one area help protect users across the entire network.

For each incoming communication, the system analyzes metadata to identify current communication patterns, validates relationship context using interaction graph and relationship fingerprints, and checks if current patterns match established patterns in relationship fingerprints. For valid relationships with new patterns, the system evaluates context for pattern evolution, and updates relationship fingerprints and aggregated patterns based on new communications. The system maintains privacy throughout by analyzing communication metadata without accessing message content. Through continuous updates and cross-service pattern sharing, it creates an evolving security framework based on relationship patterns rather than traditional authentication methods.

BRIEF DESCRIPTION OF THE DRAWING FIGURES

The accompanying drawings illustrate several embodiments of the invention and, together with the description, serve to explain the principles of the invention according to the embodiments. It will be appreciated by one skilled in the art that the particular embodiments illustrated in the drawings are merely exemplary, and are not to be considered as limiting of the scope of the invention or the claims herein in any way.

FIG. 1 is a block diagram illustrating an exemplary hardware architecture of a computing device used in an embodiment of the invention.

FIG. 2 is a block diagram illustrating an exemplary logical architecture for a client device, according to an embodiment of the invention.

FIG. 3 is a block diagram showing an exemplary architectural arrangement of clients, servers, and external services, according to an embodiment of the invention.

FIG. 4 is another block diagram illustrating an exemplary hardware architecture of a computing device used in various embodiments of the invention.

FIG. 5 is an example of communications system architecture in which a communication management server is implemented for managing security and processing of incoming communications, according to an embodiment of the invention.

FIG. 6 is an illustration of an interaction graph, according to an embodiment of the invention.

FIGS. 7A and 7B depict the dashboard of a mobile application associated with a communication management server on a user interface, according to an embodiment of the invention.

FIG. 8 illustrates an example of an environment state model used by a master AI agent for determining the action, according to an embodiment of the invention.

FIG. 9 is a flow diagram of an example method for determining an action for processing an incoming communication using an MDP model-based approach, according to an embodiment of the invention.

FIG. 10 is a flow diagram of an example method for determining an action for processing an incoming communication using model-free and model-based modes, in accordance with an embodiment of the invention.

FIG. 11 is a flow of an example method for generating relationship fingerprints, according to an embodiment of the invention.

FIG. 12 is a flow diagram of an example method for analyzing the incoming communication metadata to identify communication patterns.

FIG. 13 is a flow diagram of an example method for securely validating incoming communications using pattern-based relationship fingerprints, according to an embodiment of the invention.

DETAILED DESCRIPTION

The inventor has conceived, and reduced to practice, a security validation system using relationship fingerprints. Relationship fingerprints may be unique pattern-based identifiers that capture how two parties communicate. Unlike traditional security methods that rely on credentials or content monitoring, the relationship fingerprints based security validation uses communication patterns such as timing, channel preferences, and interaction behaviors without ever accessing message content. When an incoming communication arrives, the system first validates the relationship context using historical interaction data, then checks if current communication patterns match established relationship fingerprints. These fingerprints naturally evolve as relationships develop, becoming more sophisticated over time and makes spoofing mathematically impossible since attackers cannot fabricate the complex pattern history that defines real relationships. The system also enables cross-service pattern sharing through aggregated patterns that can be normalized and shared across networks, creating a self-strengthening security framework where attack patterns identified in one area help protect users across the entire network. This approach may provide security while maintaining complete privacy, with protection that automatically strengthens as relationships develop and pattern knowledge grows.

Headings of sections provided in this patent application and the title of this patent application are for convenience only and are not to be taken as limiting the disclosure in any way.

Devices that are in communication with each other need not be in continuous communication with each other, unless expressly specified otherwise. In addition, devices that are in communication with each other may communicate directly or indirectly through one or more communication means or intermediaries, logical or physical.

A description of an embodiment with several components in communication with each other does not imply that all

such components are required. To the contrary, a variety of optional components may be described to illustrate a wide variety of possible embodiments of one or more of the inventions and in order to fully illustrate one or more aspects of the inventions. Similarly, although process steps, method steps, algorithms or the like may be described in sequential order, such processes, methods, and algorithms may generally be configured to work in alternate orders, unless specifically stated to the contrary. In other words, any sequence or order of steps that may be described in this patent application does not, in and of itself, indicate a requirement that the steps be performed in that order. The steps of the described processes may be performed in any order practical. Further, some steps may be performed simultaneously despite being described or implied as occurring non-simultaneously (e.g., because one step is described after the other step). Moreover, the illustration of a process by its depiction in a drawing does not imply that the illustrated process is exclusive of other variations and modifications thereto, does not imply that the illustrated process or any of its steps are necessary to one or more of the invention(s), and does not imply that the illustrated process is preferred. Also, steps are generally described once per embodiment, but this does not mean they must occur once, or that they may only occur once each time a process, method, or algorithm is carried out or executed. Some steps may be omitted in some embodiments or some occurrences, or some steps may be executed more than once in a given embodiment or occurrence.

When a single device or article is described herein, it will be readily apparent that more than one device or article may be used in place of a single device or article. Similarly, where more than one device or article is described herein, it will be readily apparent that a single device or article may be used in place of more than one device or article.

The functionality or features of a device may be alternatively embodied by one or more other devices that are not explicitly described as having such functionality or features. Thus, other embodiments of one or more of the inventions need not include the device itself.

Techniques and mechanisms described or referenced herein will sometimes be described in singular form for clarity. However, it should be appreciated that particular embodiments may include multiple iterations of a technique or multiple instantiations of a mechanism unless noted otherwise. Process descriptions or blocks in figures should be understood as representing modules, segments, or portions of code that include one or more executable instructions for implementing specific logical functions or steps in the process. Alternate implementations are included within the scope of embodiments of the present invention in which, for example, functions may be executed out of order from that shown or discussed, including substantially concurrently or in reverse order, depending on the functionality involved, as would be understood by those having ordinary skill in the art.

One or more different inventions may be described in the present application. Further, for one or more of the inventions described herein, numerous alternative embodiments may be described; it should be appreciated that these are presented for illustrative purposes only and are not limiting of the inventions contained herein or the claims presented herein in any way. One or more of the inventions may be widely applicable to numerous embodiments, as may be readily apparent from the disclosure. In general, embodiments are described in sufficient detail to enable those skilled in the art to practice one or more of the inventions, and it should be appreciated that other embodiments may be

5

utilized and that structural, logical, software, electrical and other changes may be made without departing from the scope of the particular inventions. Accordingly, one skilled in the art will recognize that one or more of the inventions may be practiced with various modifications and alterations. Particular features of one or more of the inventions described herein may be described with reference to one or more particular embodiments or figures that form a part of the present disclosure, and in which are shown, by way of illustration, specific embodiments of one or more of the inventions. It should be appreciated, however, that such features are not limited to usage in the one or more particular embodiments or figures with reference to which they are described. The present disclosure is neither a literal description of all embodiments of one or more of the inventions nor a listing of features of one or more of the inventions that must be present in all embodiments.

Headings of sections provided in this patent application and the title of this patent application are for convenience only and are not to be taken as limiting the disclosure in any way.

Devices that are in communication with each other need not be in continuous communication with each other, unless expressly specified otherwise. In addition, devices that are in communication with each other may communicate directly or indirectly through one or more communication means or intermediaries, logical or physical.

A description of an embodiment with several components in communication with each other does not imply that all such components are required. To the contrary, a variety of optional components may be described to illustrate a wide variety of possible embodiments of one or more of the inventions and in order to more fully illustrate one or more aspects of the inventions. Similarly, although process steps, method steps, algorithms or the like may be described in a sequential order, such processes, methods and algorithms may generally be configured to work in alternate orders, unless specifically stated to the contrary. In other words, any sequence or order of steps that may be described in this patent application does not, in and of itself, indicate a requirement that the steps be performed in that order. The steps of described processes may be performed in any order practical. Further, some steps may be performed simultaneously despite being described or implied as occurring non-simultaneously (e.g., because one step is described after the other step). Moreover, the illustration of a process by its depiction in a drawing does not imply that the illustrated process is exclusive of other variations and modifications thereto, does not imply that the illustrated process or any of its steps are necessary to one or more of the invention(s), and does not imply that the illustrated process is preferred. Also, steps are generally described once per embodiment, but this does not mean they must occur once, or that they may only occur once each time a process, method, or algorithm is carried out or executed. Some steps may be omitted in some embodiments or some occurrences, or some steps may be executed more than once in a given embodiment or occurrence.

When a single device or article is described herein, it will be readily apparent that more than one device or article may be used in place of a single device or article. Similarly, where more than one device or article is described herein, it will be readily apparent that a single device or article may be used in place of the more than one device or article.

The functionality or the features of a device may be alternatively embodied by one or more other devices that are not explicitly described as having such functionality or

6

features. Thus, other embodiments of one or more of the inventions need not include the device itself.

Techniques and mechanisms described or referenced herein will sometimes be described in singular form for clarity. However, it should be appreciated that particular embodiments may include multiple iterations of a technique or multiple instantiations of a mechanism unless noted otherwise. Process descriptions or blocks in figures should be understood as representing modules, segments, or portions of code which include one or more executable instructions for implementing specific logical functions or steps in the process. Alternate implementations are included within the scope of embodiments of the present invention in which, for example, functions may be executed out of order from that shown or discussed, including substantially concurrently or in reverse order, depending on the functionality involved, as would be understood by those having ordinary skill in the art.

Hardware Architecture

Generally, the techniques disclosed herein may be implemented on hardware or a combination of software and hardware. For example, they may be implemented in an operating system kernel, in a separate user process, in a library package bound into network applications, on a specially constructed machine, on an application-specific integrated circuit (ASIC), or on a network interface card.

Software/hardware hybrid implementations of at least some of the embodiments disclosed herein may be implemented on a programmable network-resident machine (which should be understood to include intermittently connected network-aware machines) selectively activated or reconfigured by a computer program stored in memory. Such network devices may have multiple network interfaces that may be configured or designed to utilize different types of network communication protocols. A general architecture for some of these machines may be described herein in order to illustrate one or more exemplary means by which a given unit of functionality may be implemented. According to specific embodiments, at least some of the features or functionalities of the various embodiments disclosed herein may be implemented on one or more general-purpose computers associated with one or more networks, such as for example an end-user computer system, a client computer, a network server or other server system, a mobile computing device (e.g., tablet computing device, mobile phone, smartphone, laptop, or other appropriate computing device), a consumer electronic device, a music player, or any other suitable electronic device, router, switch, or other suitable device, or any combination thereof. In at least some embodiments, at least some of the features or functionalities of the various embodiments disclosed herein may be implemented in one or more virtualized computing environments (e.g., network computing clouds, virtual machines hosted on one or more physical computing machines, or other appropriate virtual environments).

Referring now to FIG. 1, there is shown a block diagram depicting an exemplary computing device **100** suitable for implementing at least a portion of the features or functionalities disclosed herein. Computing device **100** may be, for example, any one of the computing machines listed in the previous paragraph, or indeed any other electronic device capable of executing software- or hardware-based instructions according to one or more programs stored in memory. Computing device **100** may be adapted to communicate with a plurality of other computing devices, such as clients or servers, over communications networks such as a wide area network a metropolitan area network, a local area network,

a wireless network, the Internet, or any other network, using known protocols for such communication, whether wireless or wired.

In one embodiment, computing device **100** includes one or more central processing units (CPU) **102**, one or more interfaces **110**, and one or more busses **106** (such as a peripheral component interconnect (PCI) bus). When acting under the control of appropriate software or firmware, CPU **102** may be responsible for implementing specific functions associated with the functions of a specifically configured computing device or machine. For example, in at least one embodiment, a computing device **100** may be configured or designed to function as a server system utilizing CPU **102**, local memory **101** and/or remote memory **120**, and interface(s) **110**. In at least one embodiment, CPU **102** may be caused to perform one or more of the different types of functions and/or operations under the control of software modules or components, which for example, may include an operating system and any appropriate applications software, drivers, and the like.

CPU **102** may include one or more processors **103** such as, for example, a processor from one of the Intel, ARM, Qualcomm, and AMD families of microprocessors. In some embodiments, processors **103** may include specially designed hardware such as application-specific integrated circuits (ASICs), electrically erasable programmable read-only memories (EEPROMs), field-programmable gate arrays (FPGAs), and so forth, for controlling operations of computing device **100**. In a specific embodiment, a local memory **101** (such as non-volatile random-access memory (RAM) and/or read-only memory (ROM), including for example one or more levels of cached memory) may also form part of CPU **102**. However, there are many different ways in which memory may be coupled to system **100**. Memory **101** may be used for a variety of purposes such as, for example, caching and/or storing data, programming instructions, and the like. It should be further appreciated that CPU **102** may be one of a variety of system-on-a-chip (SOC) type hardware that may include additional hardware such as memory or graphics processing chips, such as a Qualcomm SNAPDRAGON™ or Samsung EXYNOS™ CPU as are becoming increasingly common in the art, such as for use in mobile devices or integrated devices.

As used herein, the term “processor” is not limited merely to those integrated circuits referred to in the art as a processor, a mobile processor, or a microprocessor, but broadly refers to a microcontroller, a microcomputer, a programmable logic controller, an application-specific integrated circuit, and any other programmable circuit.

In one embodiment, interfaces **110** are provided as network interface cards (NICs). Generally, NICs control the sending and receiving of data packets over a computer network; other types of interfaces **110** may for example support other peripherals used with computing device **100**. Among the interfaces that may be provided are Ethernet interfaces, frame relay interfaces, cable interfaces, DSL interfaces, token ring interfaces, graphics interfaces, and the like. In addition, various types of interfaces may be provided such as, for example, universal serial bus (USB), Serial, Ethernet, FIREWIRE™, THUNDERBOLT™, PCI, parallel, radio frequency (RF), BLUETOOTH™, near-field communications (e.g., using near-field magnetics), 802.11 (Wi-Fi), frame relay, TCP/IP, ISDN, fast Ethernet interfaces, Gigabit Ethernet interfaces, Serial ATA (SATA) or external SATA (ESATA) interfaces, high-definition multimedia interface (HDMI), digital visual interface (DVI), analog or digital audio interfaces, asynchronous transfer mode (ATM) inter-

faces, high-speed serial interface (HSSI) interfaces, Point of Sale (POS) interfaces, fiber data distributed interfaces (FDDIs), and the like. Generally, such interfaces **110** may include physical ports appropriate for communication with appropriate media. In some cases, they may also include an independent processor (such as a dedicated audio or video processor, as is common in the art for high-fidelity A/V hardware interfaces) and, in some instances, volatile and/or non-volatile memory (e.g., RAM).

Although the system shown in FIG. 1 illustrates one specific architecture for a computing device **100** for implementing one or more of the inventions described herein, it is by no means the only device architecture on which at least a portion of the features and techniques described herein may be implemented. For example, architectures having one or any number of processors **103** may be used, and such processors **103** may be present in a single device or distributed among any number of devices. In one embodiment, a single processor **103** handles communications as well as routing computations, while in other embodiments a separate dedicated communications processor may be provided. In various embodiments, different types of features or functionalities may be implemented in a system according to the invention that includes a client device (such as a tablet device or smartphone running client software) and server systems (such as a server system described in more detail below).

Regardless of network device configuration, the system of the present invention may employ one or more memories or memory modules (such as, for example, remote memory block **120** and local memory **101**) configured to store data, program instructions for the general-purpose network operations, or other information relating to the functionality of the embodiments described herein (or any combinations of the above). Program instructions may control the execution of or comprise an operating system and/or one or more applications, for example. Memory **120** or memories **101**, **120** may also be configured to store data structures, configuration data, encryption data, historical system operations information, or any other specific or generic non-program information described herein.

Because such information and program instructions may be employed to implement one or more systems or methods described herein, at least some network device embodiments may include non-transitory machine-readable storage media, which, for example, may be configured or designed to store program instructions, state information, and the like for performing various operations described herein. Examples of such non-transitory machine-readable storage media include, but are not limited to, magnetic media such as hard disks, floppy disks, and magnetic tape; optical media such as CD-ROM disks; magneto-optical media such as optical disks, and hardware devices that are specially configured to store and perform program instructions, such as read-only memory devices (ROM), flash memory (as is common in mobile devices and integrated systems), solid state drives (SSD) and “hybrid SSD” storage drives that may combine physical components of solid state and hard disk drives in a single hardware device (as are becoming increasingly common in the art with regard to personal computers), memristor memory, random access memory (RAM), and the like. It should be appreciated that such storage means may be integral and non-removable (such as RAM hardware modules that may be soldered onto a motherboard or otherwise integrated into an electronic device), or they may be removable such as swappable flash memory modules (such as “thumb drives” or other removable media designed for

rapidly exchanging physical storage devices), “hot-swappable” hard disk drives or solid state drives, removable optical storage discs, or other such removable media, and that such integral and removable storage media may be utilized interchangeably. Examples of program instructions

include both object code, such as may be produced by a compiler, machine code, such as may be produced by an assembler or a linker, byte code, such as may be generated by for example a Java™ compiler and may be executed using a Java virtual machine or equivalent, or files containing higher level code that may be executed by the computer using an interpreter (for example, scripts written in Python, Perl, Ruby, Groovy, or any other scripting language).

In some embodiments, systems according to the present invention may be implemented on a standalone computing system. Referring now to FIG. 2, there is shown a block diagram depicting a typical exemplary architecture of one or more embodiments or components thereof on a standalone computing system. Computing device 200 includes processors 210 that may run software that carry out one or more functions or applications of embodiments of the invention, such as for example a client application 230. Processors 210 may carry out computing instructions under control of an operating system 220 such as, for example, a version of Microsoft’s WINDOWS™ operating system, Apple’s Mac OS/X or iOS operating systems, some variety of the Linux operating system, Google’s ANDROID™ operating system, or the like. In many cases, one or more shared services 225 may be operable in system 200, and may be useful for providing common services to client applications 230. Services 225 may for example be WINDOWS™ services, user-space common services in a Linux environment, or any other type of common service architecture used with operating system 210. Input devices 270 may be of any type suitable for receiving user input, including for example a keyboard, touchscreen, microphone (for example, for voice input), mouse, touchpad, trackball, or any combination thereof. Output devices 260 may be of any type suitable for providing output to one or more users, whether remote or local to system 200, and may include for example one or more screens for visual output, speakers, printers, or any combination thereof. Memory 240 may be random-access memory having any structure and architecture known in the art, for use by processors 210, for example, to run software. Storage devices 250 may be any magnetic, optical, mechanical, memristor, or electrical storage device for storage of data in digital form (such as those described above, referring to FIG. 1). Examples of storage devices 250 include flash memory, magnetic hard drive, CD-ROM, and/or the like.

In some embodiments, systems of the present invention may be implemented on a distributed computing network, such as one having any number of clients and/or servers. Referring now to FIG. 3, there is shown a block diagram depicting an exemplary architecture 300 for implementing at least a portion of a system according to an embodiment of the invention on a distributed computing network. According to the embodiment, any number of clients 330 may be provided. Each client 330 may run software for implementing client-side portions of the present invention; clients may comprise a system 200 such as that illustrated in FIG. 2. In addition, any number of servers 320 may be provided for handling requests received from one or more clients 330. Clients 330 and servers 320 may communicate with one another via one or more electronic networks 310, which may be in various embodiments any of the Internet, a wide area network, a mobile telephony network (such as CDMA or GSM cellular networks), a wireless network (such as Wi-Fi,

WiMAX, LTE, and so forth), or a local area network (or indeed any network topology known in the art; the invention does not prefer any one network topology over any other). Networks 310 may be implemented using any known network protocols, including for example wired and/or wireless protocols.

In addition, in some embodiments, servers 320 may call external services 370 when needed to obtain additional information, or to refer to additional data concerning a particular incoming communication. Communications with external services 370 may take place, for example, via one or more networks 310. In various embodiments, external services 370 may comprise web-enabled services or functionality related to or installed on the hardware device itself. For example, in an embodiment where client applications 230 are implemented on a smartphone or other electronic device, client applications 230 may obtain information stored in a server system 320 in the cloud or on an external service 370 deployed on one or more of a particular enterprise or user’s premises.

In some embodiments of the invention, clients 330 or servers 320 (or both) may make use of one or more specialized services or appliances that may be deployed locally or remotely across one or more networks 310. For example, one or more databases 340 may be used or referred to by one or more embodiments of the invention. It should be understood by one having ordinary skill in the art that databases 340 may be arranged in a wide variety of architectures and using a wide variety of data access and manipulation means. For example, in various embodiments, one or more databases 340 may comprise a relational database system using a structured query language (SQL), while others may comprise an alternative data storage technology such as those referred to in the art as “NoSQL” (for example, Hadoop Cassandra, Google Big Table, Mongo, and so forth). In some embodiments, variant database architectures such as column-oriented databases, in-memory databases, clustered databases, distributed databases, or even flat file data repositories may be used according to the invention. In addition, Graph-oriented databases, also known as graph databases, are designed to manage and store data structured as graphs, where entities (nodes) are interconnected with relationships (edges), examples include (Amazon Neptune, Microsoft Azure Cosmos DB, JanusGraph, TigerGraph, GraphDB and so forth). These databases are particularly effective for applications involving complex relational queries and traversals, such as social networks, recommendation systems, and network topology analysis.

In addition, vector databases also referred to as vector search databases or similarity search databases, are engineered to index, manage, and retrieve high-dimensional vectors typically generated by machine learning models. These databases are adept at handling operations such as nearest neighbor search in vector space, which is critical for tasks involving image recognition, natural language processing, and recommendation engines, where items are represented as vectors in a multi-dimensional space. Notable examples include Pinecone, Milvus, Weaviate, and Elasticsearch with vector plugins. Vector databases excel in scenarios that require matching patterns or finding similar items based on vector proximity, making them indispensable for modern AI-driven applications such as semantic search, personalization features, and fraud detection systems.

It will be appreciated by one having ordinary skill in the art that any combination of known or future database technologies may be used as appropriate unless a specific database technology or a specific arrangement of compo-

nents is specified for a particular embodiment herein. Moreover, it should be appreciated that the term “database” as used herein may refer to a physical database machine, a cluster of machines acting as a single database system, or a logical database within an overall database management system. Unless a specific meaning is specified for a given use of the term “database,” it should be construed to mean any of these senses of the word, all of which are understood as a plain meaning of the term “database” by those having ordinary skill in the art.

Similarly, most embodiments of the invention may make use of one or more security systems **360** and configuration systems **350**. Security and configuration management are common information technology (IT) and web functions, and some amount of each is generally associated with any IT or web systems. It should be understood by one having ordinary skill in the art that any configuration or security subsystems known in the art now or in the future may be used in conjunction with embodiments of the invention without limitation unless a specific security **360** or configuration system **350** or approach is specifically required by the description of any specific embodiment.

FIG. 4 shows an exemplary overview of a computer system **400** as may be used in any of the various locations throughout the system. It is exemplary of any computer that may execute code to process data. Various modifications and changes may be made to computer system **400** without departing from the broader spirit and scope of the system and method disclosed herein.

CPU **401** is connected to bus **402**, to which bus is also connected memory **403**, nonvolatile memory **404**, display **407**, I/O unit **408**, and network interface card (NIC) **413**. I/O unit **408** may, typically, be connected to keyboard **409**, pointing device **410**, hard disk **412**, and real-time clock **411**. NIC **413** connects to network **414**, which may be the Internet or a local network, which local network may or may not have connections to the Internet. Also shown as part of system **400** is power supply unit **405** connected, in this example, to ac supply **406**. Not shown are batteries that could be present, and many other devices and modifications that are well known but do not apply to the specific novel functions of the current system and method disclosed herein. It should be appreciated that some or all components illustrated may be combined, such as in various integrated applications (for example, Qualcomm or Samsung SOC-based devices), or whenever it may be appropriate to combine multiple capabilities or functions into a single hardware device (for instance, in mobile devices such as smartphones, video game consoles, in-vehicle computer systems such as navigation or multimedia systems in automobiles, or other integrated hardware devices).

In various embodiments, functionality for implementing systems or methods of the present invention may be distributed among any number of client and/or server components. For example, various software modules may be implemented for performing various functions in connection with the present invention, and such modules may be variously implemented to run on server and/or client components. Detailed Conceptual Architecture

FIG. 5 is an example of a communication system architecture **500** in which a communication management server **506** may be implemented for managing security and processing of incoming communications, according to an embodiment of the invention. In an embodiment, communication management server **506** is in communication with the multimedia gateway **508**, which in turn may be con-

nected through one or more networks **504** with a plurality of user devices **502A**, **502B** . . . **502N**, (collectively referred to as user devices **502**).

In an embodiment, user devices **502** may include one or more mobile devices, smartphones, general-purpose computers, tablet computers, laptop computers, smart wearable devices, voice command devices, Internet-of-Things (IoT) devices, or any other device capable of communicating with the network **504**, including mobile service provider **528**, and ultimately communicating through the multimedia gateway **508** with one or more components of the communication management server **506**. One subset of the user devices **502** are the devices owned and used by the registered users which are being monitored for incoming communications by the multimedia gateway **508**. Another subset of the user devices may be owned and used by known contacts of the registered users. The remainder may be user devices of users unknown to the registered user of the system.

In an embodiment, a device owner first registers an account with the communication management server **506** for their devices, thereby becoming a registered user, and sets initial preferences, regarding contacts and hours and the user's goals if the user wishes. In an embodiment, registered user devices **502** may connect using a progressive web application that uses WebRTC for communications data exchange and registration. Further, the user may answer a series of questions that set their initial objective reward function based on a holistic view of their lifestyle and short-term and long-term goals. After the registered user is set up, incoming communications to the registered user's devices are handled by communication management server **506**.

In an embodiment, communication management server **506** may include a processor **511** and a plurality of programming instructions stored in a memory **512**. The instruction when executed may be configured to manage incoming communications for registered user devices. Communication management server **506** may be configured to communicate with user devices **502** via the multimedia gateway **508**, which may serve as an intermediary between one or more networks **504** and communication management server **506**.

In an embodiment, multimedia gateway **508** receives an incoming communication notification from the communication networks **504** including signaling and routing information as well as originating and terminating user identification information from among the plurality of user devices **502**. Multimedia gateway **508** responds with appropriate information to pause the communications process while it is processed by communication management server **506**. Multimedia gateway **508** may be configured to communicate signaling and routing information along with originating and destination user identification information from among a plurality of users **502** associated with an individual incoming communication from an originating network **504**.

In an embodiment, multimedia gateway **508** may be configured to make use of existing third-party attestation data from a third-party attestation service **540** if available. This brings additional contextual data relating to a telco customer and the source of the incoming call. This additional data may be added to the interaction graph **520** to further improve the decisions made by the master AI agent **518** when handling the incoming call.

In an embodiment, user preferences **524** may be maintained by communication management server **506** and may include: Do not disturb (DND) hours (set unavailable times to mute notifications), contact exceptions (override hours for

13

priority contacts), channel priorities—rank messaging, email, calls for importance, custom rules (If-then logic to route senders and keywords), AI delegation (desired level of automation versus user confirmation), activity settings (visibility of transcripts, recording, data), integration permissions (allowed 3rd parties and data access). In addition, the registered user can enter a statement of their overall short-term and long-term goals relating to communications decisions. Details related to configuring the user preferences and rules are described in FIGS. 7A and 7B.

In an embodiment, the interaction graph 520 may be a heterogeneous multigraph holding multiple graph representations with different types of nodes and different types of edges between nodes for different purposes. At the lowest level, the interaction graph 520 may capture current or historical communications (or communication attempts) between devices where each node represents a unique device, and each edge represents an individual communication between devices. Each node has a rich set of attributes relating to the details of the device and each edge has a rich set of edge attributes associated with the communication between the nodes, including information like the channel type, duration, subjects, content, and embeddings of content. In an embodiment, these detailed historical interaction data are essentially immutable (as they can be considered the raw “facts” in the terminology of data warehousing).

The interaction graph 520 in the communication management system is a complex network of nodes and edges representing users, their relationships, and communication interactions. To effectively leverage this graph-structured data for decision-making, the master AI agent 518 employs Graph Neural Networks (GNNs), a cutting-edge AI technique designed to process and learn from graphs.

GNNs work by iteratively updating the representation (embedding) of each node and edge in the graph based on the features of the node itself, the features of its neighboring nodes, and the features of the connecting edges. This allows the GNN to capture both the local and global structure of the interaction graph and learn meaningful patterns and relationships.

The master AI agent 518 uses a specific type of GNN called a Graph Convolutional Network (GCN) to process the interaction graph. The GCN consists of multiple layers, each of which performs the following operations:

1. Feature Aggregation: For each node, the GCN aggregates the features of its neighboring nodes, and the features of the connecting edges, weighted by the strength of their connections (edges).
2. Edge Feature Transformation: The features of the edges are transformed using a learnable weight matrix to compute new edge representations that capture the characteristics of the relationships between nodes.
3. Node Feature Transformation: The aggregated node features and transformed edge features are then combined and transformed using another learnable weight matrix to compute a new representation for the node.
4. Non-linear Activation: A non-linear activation function, such as ReLU (Rectified Linear Unit), is applied to the transformed features to introduce non-linearity and increase the expressive power of the GCN.

By stacking multiple GCN layers, the master AI agent can learn hierarchical representations of the interaction graph, capturing both low-level features (e.g., individual communication interactions and relationship characteristics) and high-level patterns (e.g., group communication dynamics and complex relationship structures).

14

The learned representations from the GCN, including both node and edge embedding, are then used as input features for the Environment State Model 522 and a Markov Decision Process (MDP) which is then solved to find the Action Selection Function (ASF) 510 to make more informed and context-aware decisions for managing incoming communications based on the user’s preferences and the structure of the interaction graph, and the nature of the relationships between users. Specifically, the learned embeddings from the GCN may be used as input features for the MDP in the following ways:

State Representation: The Environment State Model 522 of the MDP may be represented using the embeddings of the relevant nodes and edges in the interaction graph. For example, when a new communication arrives, the state may be constructed by concatenating the embeddings of the sender node, the recipient node, and the edge representing their relationship. This environment state representation provides the MDP with rich information about the context of the communication, including the characteristics of the involved users and their relationship. Clustering in the embedding space may then be performed to proactively manage the size of the state space.

Reward Function: The reward function of the MDP is designed to align with the user’s preferences and the desired communication management objectives. The GCN embeddings may be used to compute rewards based on the characteristics of the users their relationships and the communication content. For example, edge embeddings can assign higher rewards for prioritizing communications from important contacts, or the importance or urgency of the communication based on the nature of the relationship between the sender and the recipient and the embedded content of the communications.

Transition Probabilities: The transition probabilities of the MDP, which define the likelihood of moving from one state to another based on the chosen action, can be based on historical frequency or in their absence, also estimated using the GCN embeddings. The embeddings can capture patterns and dynamics in the interaction graph that influence the outcomes of communication management decisions. For instance, the node and edge embeddings can help predict how users are likely to respond to different actions taken by the master AI agent, such as the probability of a delegated task being completed successfully based on the characteristics of the assigned person or agent.

At the higher level, the interaction graph may capture relationships between the parties (who operate the devices) where each node represents a unique party, and where each edge between parties represents a relationship between parties. This enables the interaction graph to represent affinities, common interests, and other associations between parties as general relationships. These relationships may be dynamic, unlike the immutable device-level graph edges. Each node may have a rich set of known attributes about the party, e.g. for human parties this this can include demographics, personal preferences, and interests. But there can be multiple kinds of relationships between two parties. One kind of relationship can be e.g. a declared actual familial or business relationship between parties, e.g. “is the spouse of” “is a child of” or “is business partner of” Another relationship could be an “affiliation” between two parties in terms of a formal or informal connection or association that would typically involve some form of cooperative relationship or

15

alignment or common interest, e.g. “share interest in mountain biking” or “share interest in antique collection.” A communication interaction relationship exists when there is frequent communication between parties even though the topic of conversation may not be known. The interaction graph therefore uses an aggregated measure of frequency or strength of communications between two parties to represent a communications interaction relationship.

The interaction graph 520 discovers and stores the association of one or more devices to each party over time or also the fact that multiple parties may share the use of certain devices in some circumstances (such as a shared home or office communications device). In an embodiment, the parties included in the interaction graph nodes also include AI machines acting like proxies for humans which may include the AI communication agents 516 of the system itself, but also any third-party communication bots.

Interaction graph 520 therefore captures relationships and associations between devices and parties that may be useful in optimizing communication interactions and unlocking value through actions creating new interactions (or blocking interactions) between nodes. Data from interaction graph 520 may be used for additional feature calculations such as correlation, causation, covariance, association rules, dependencies, time-series relationships, spatial and temporal associations, and general multivariate relationships. A simple example of an interaction graph 520 is described in FIG. 6.

In an embodiment, master AI agent 518 receives signaling and routing information about incoming communication directed toward a first-user device via multimedia gateway 508. Based on the content and context of the incoming communication, the master AI agent 518 first updates the interaction graph 520 and then uses the interaction graph 520 data to update environment state model 522 in case a similar state is not previously captured.

In an embodiment, environment state model 522 is a state machine that tracks the current environment state of the registered user which may include the number and type of live communications the registered user is currently paying attention to, including the state of each. Further in some cases, in addition to considering the communication state of the registered user other attributes of the physical environment, the user is in, and even states such as the emotional, and tiredness state of the registered user may be tracked by the environment model.

Environment state model 522 may be used by master AI agent 518 to determine the action to be performed for processing the incoming communication. Environment state model 522 may be the basis of a reinforcement learning-based AI model that defines states and associated actions based on learning and user preferences 524. In an embodiment, environment state model 522 is designed to allow communication management server 506 to identify and trigger the modification of the communication state of user devices and to take other actions. In an embodiment, environment state model 522 may be used by the master AI agent 518 to implement a fully observable Markov Decision Process (MDP). Details of an environment state model 522 are described in conjunction with FIG. 8.

In some aspects, the techniques described herein relate to a model-based method for managing incoming communications, whereby the environment state model 522 is selected to be partially observable and may be modelled by a Partially Observable Markov Decision Process (POMDP). The POMDP allows the invention to plan communication actions under uncertainty that considers not just the proba-

16

bilistic outcomes of actions (like for MDPs) but also the uncertainty of what the current state is due to privacy or other data restrictions.

In an embodiment, master AI agent 518 may be configured to initiate actions for incoming communication. The actions taken on behalf of the user may be in alignment with the user’s short-term and long-term goals. In one embodiment, the master AI agent 518 may perform decision-making using two different approaches to reinforcement learning referred to as “model-based” and “model-free.” The master AI agent 518 may switch between these two approaches based on the availability of adequate relevant historical data, the observed decision quality, user goal modifications, or changes in the user’s environment. The terms “model-based” and “model-free” are used here in the context understood by those skilled in the art of reinforcement learning. Model-free approach to decision-making and switching between model-based and model-free approaches is described in FIG. 10. FIGS. 6-9 describe the use of a model-based approach for determining actions for processing incoming communication.

During operation, based on the information available in the system, a current state of the environment is generated and master AI agent 518 may invoke the action selection function (ASF) 510 to determine the action to take to process the incoming communication. The action selected by ASF 510 may be executed by the master AI agent 518 using the multimedia gateway 508 and/or AI communication agent. For example, the action may be to send an instruction directly to the multimedia gateway 508 to block the incoming call, or the master AI agent 518 invoke one or more generative AI communication agents 516 to perform additional sub-tasks according to different agentic roles before sending an instruction to the multimedia gateway 508.

For some incoming communications, the action chosen by the ASF 510 and executed by the master AI agent 518 may be a complex one requiring the master AI agent 518 to invoke an AI communication agent 516.

In an embodiment, AI communication agents 516 are generative AI models that use natural language processing to extract preferences from conversations, emails, and messaging history to infer optimal communication hours, interests for conversation topics, and priority contacts. In an embodiment, AI communication agents 516 operate in the framework of retrieval augmented generation, declarative model building, and collective reasoning under the control of the master AI agent 518.

The AI agents in the communication management system are more than simple generative AI agents and may utilize Machine Learning (ML) techniques to continuously improve their decision-making capabilities. This involves:

Data Collection: Gathering historical data on incoming communications, user preferences, actions taken, and outcomes.

Feature Extraction: Identifying key features from the collected data that are relevant for decision-making, such as sender attributes, communication type, content keywords, and user feedback.

Model Training: Using the extracted features to train ML models, such as decision trees, support vector machines, deep neural networks, or graph neural networks to predict the best actions for handling incoming communications.

Model Evaluation: Testing the trained models on a separate dataset to assess their performance and make any necessary adjustments.

Online Learning: Continuously updating the ML models based on new data and user feedback to adapt to changing user preferences and communication patterns.

Unsupervised Learning: Applying techniques such as clustering and dimensionality reduction to identify hidden patterns and structures within the data, which can provide insights and improve the decision-making process.

By leveraging ML techniques, AI agents can learn from past experiences and improve their ability to make optimal decisions that align with the user's goals and preferences.

In an embodiment, different AI communication agents **516** may be configured to support different types of communications. For example, a first AI communication agent may be associated with performing voice-based actions, a second AI communication agent may be associated with handling email communication and a third AI communication agent may be associated with incoming text messages.

During the processing of the incoming communication, an AI communication agent **516** may be utilized by master AI agent **518** to generate communication contents for the incoming communication. In some cases, AI communication agents **516** may analyze communication content.

In addition to machine learning and generative AI techniques, AI communication agents may employ more conventional Natural Language Processing (NLP) techniques to analyze and understand the content of incoming communications. This includes:

Tokenization: Breaking down the text into individual words or tokens.

Part-of-Speech (POS) tagging: Identifying the grammatical role of each word (e.g., noun, verb, adjective) to understand the structure of the sentences.

Named Entity Recognition (NER): Identifying and classifying named entities such as people, organizations, locations, and dates.

Sentiment Analysis: Determining the sentiment (positive, negative, or neutral) expressed in the communication to gauge the emotional tone.

Intent Recognition: Identifying the purpose or goal of the communication, such as making a request, asking a question, or expressing a complaint.

By applying these NLP techniques, the AI communication agents can extract meaningful information from the incoming communications, which is then used by the master AI agent to make informed decisions on how to handle each communication based on the user's preferences and the interaction graph.

In an embodiment, communication management server **506** may include a blacklist/whitelist data **514**. This may be used to avoid unnecessary repeating of reasoning to determine whether an incoming multimedia communication should be immediately blocked or be connected for further processing. The data may also be prepopulated through the 3rd party attestation service or user preferences. The data may be configured with an expiry timeout.

In an embodiment, communication management server **506** may include a vector DB **526** which is a specialized type of database designed to efficiently handle vector embeddings, which are high-dimensional vectors used to represent complex data items in a form suitable for machine learning algorithms, particularly in the context of similarity search operations or compressed representations of graph data. This may be used by the master AI agent **518**, in its method of

managing the large state-space during model-based operation, but also for more efficient routing and semantic search operations.

During operation, an incoming communication associated with a user device among a plurality of user devices is received on a multimedia gateway wherein the incoming communication is one of a voice message, an audio call, a text message, an email, or a video communication. The multimedia gateway provides new context and content data associated with the incoming communication which is sent to the master AI agent. The master AI agent stores the new information in the interaction graph memory. It then invokes the reinforcement learning action selection function ASF to determine an action for processing the incoming communication, wherein (for the model-based mode) the action is associated with the modified environment state, and information from the interaction graph, wherein the interaction graph is indicative of previous interactions and relationships of the user and user device, other user devices, and also with AI communication agents invoked previously by the communication management server.

The action selected by the action selection function ASF may be executed by a function call from the master AI agent **518** to an internal application programming interface (API). Some function calls are executed on the multimedia gateway, but a selected action may require the master AI agent to perform additional sub-tasks including the selection and configuration of AI communication agents to co-operate to further analyze content, or for generating new multimedia content, perform additional limited reasoning and estimations and so on. On completion of such sub-tasks or in the event of a timeout, the master AI agent **518** may send a completion message back to the multimedia gateway **508** that specifies any final external actions for multimedia gateway **508** to continue handling user devices.

In an embodiment, multimedia gateway **508** and communication management server **506** may operate in the cloud, and master AI agent **518** may perform functions when one or even all of the registered user's devices are offline or even powered down. This enables a high degree of intelligent communication handling, content analysis, and generation to occur even in the complete absence of the registered user and their devices.

In an embodiment, communication management server **506** may be implemented as a set of microservices for each of the various components of communication management server **506** running in containers. In an embodiment, the multimedia gateway **508** may be software that sits in front of an application programming interface (API) or group of microservices to facilitate requests and delivery of data and services between user devices **502** and the communication management server **506**. Its primary role may be to act as a single-entry point and standardized process for interactions between the communication management server **506**, and the external networks **504**, user devices **502**, and other external devices.

In another embodiment, the components of the communication management server **506** and multimedia gateway **508** may be implemented as containerized microservices that run in a cloud computing environment. The use of containerization is important in that service instances can be spun up in near-real-time and then made inactive when no longer required. Certain elements such as the multimedia gateway **508** need to be running continuously but can be shared through multi-tenant configuration. The architecture is provided with telco-grade security with encryption, data isolation, access controls, key management, and auditing.

Communications are encrypted in transit and at rest using TLS, HTTPS, and AES standards. AI communication agents **516** processing may be isolated through containerization and private cloud tenants. Granular role-based access control may be used for managing user and agent permissions. Secrets and keys follow principles of least privilege and rotation. Detailed security event logging provides transparency and supports forensic analysis. User authentication may include Password, 2FA, SSO, and biometrics.

In an embodiment, communication management server **506** includes a pattern-based decision manager **532** for security validation of the incoming communication. Pattern-based decision manager **532** manages relationship validation, performs pattern-based security checks and master AI agent **518** determines communication actions.

In an embodiment, pattern-based decision manager **532** uses a pattern analysis engine **536** to identify communication patterns and extract pattern characteristics.

Communication patterns are indicative of how users interact with each other. Communication patterns for every relationship (interaction between two users) may include, but is not limited to, timing of responses, preferred communication channels, and frequency of contact. Communication patterns naturally evolve and become more sophisticated as relationships develop, creating unique fingerprints.

Pattern analysis engine **536** may analyze communication metadata to determine communication patterns between the parties. The parties are a user device associated with the incoming call and the first-user device which is the recipient of the incoming communication.

In an embodiment, the communication patterns may be temporal patterns that identify the time when communications occur, the frequency of interactions between the user devices, the duration of communication, and intervals between interactions. In some cases, the frequency of interaction may be used for tracking trends including, but not limited to, increasing or decreasing interaction frequency, patterns in communication volume, and variations across different time periods. Further, pattern analysis engine **536** may track these temporal patterns to identify characteristics such as response times, preferred communication hours, and changes in these temporal patterns over time.

In an embodiment, the communication patterns may be channel patterns that identify the preferred communication channels (voice, text, email, etc.) between the parties. Further, pattern analysis engine **536** may track shifts in channel preferences over time.

In an embodiment, the communication patterns may be response behavior patterns that identify the time taken by parties to respond to each other, the consistency of responses, and any changes in response patterns that might indicate shifts in relationship dynamics.

In an embodiment, memory **512** may be configured to store an interaction graph **520**, an environment state model **522** for the registered user for model-based mode, and user preferences **524**. In addition to interaction graph **520**, the memory **512** may include a relationship fingerprints database **534**, and a pattern database **538**.

In an embodiment interaction graph **520** may provide a relationship context to pattern analysis engine **536**. In an embodiment, the communication patterns identified by pattern analysis engine **536** may be used for updating interaction graph **520**. For example, new nodes may be added in case of receiving incoming communication from new user devices. Edge weights are updated based on interaction frequency and a relationship context is created/strengthened. In an embodiment, interaction graph **520** may further be

updated based on the communication channel used. Interaction graph **520** provides relationship context for communication between parties.

In an embodiment, relationship fingerprints stored at relationship fingerprints database **534** may provide the unique characteristics of communication relationship between two parties and it may include temporal patterns (timing, frequency, duration), channel preferences patterns (voice, text, email), response behaviors (speed, consistency), engagement levels, and relationship context. In an embodiment, relationship fingerprints database **534** implements a hierarchical structure that organizes communication patterns at multiple levels—individual relationships, organizational relationships, and network-wide patterns. Each level maintains its own set of pattern characteristics while enabling cross-referencing for pattern validation. The database architecture supports real-time pattern matching while continuously incorporating new pattern observations, creating an evolving knowledge base of communication behaviors.

Relationship fingerprints may be generated from interaction graph **520** data and communication patterns. Relationship fingerprints database **534** may be constantly updated with the processing of new incoming communications. Relationship fingerprints offer relationship characteristics and may be indicative of established communication behaviors.

The fingerprint continuously evolves as the relationship develops, making it increasingly precise in distinguishing legitimate communications from unwanted ones. Each relationship generates its own unique fingerprint that adapts over time, enabling sophisticated pattern matching while preserving complete privacy since no message content is ever accessed or stored.

In an embodiment, communication management server **506** stores aggregated patterns used across services and networks in pattern database **538**. Details related to the generation of aggregated patterns using cross-network analysis are described in in FIG. 12.

In an embodiment, pattern-based decision manager **532** may use information available in the relationship fingerprints database **534** to process the incoming communication.

During operation, when an incoming communication is received, pattern analysis engine **536** may analyze the communication metadata without accessing content to identify current communication patterns. These patterns are checked against relationship fingerprints stored in a database that captures unique characteristics of how parties communicate. The validation process involves first verifying the relationship context using historical interaction data from an interaction graph, and then determining if current communication patterns match established patterns in the relationship fingerprints. For valid relationships without matching patterns, the system evaluates if there is legitimate context for a new pattern and updates the relationship fingerprints accordingly. Pattern-based decision manager **532** continuously strengthens security by updating relationship fingerprints with new patterns and sharing validated patterns across services, enabling protection that evolves with relationships while maintaining privacy.

Details related to the security validation of incoming communication using relationship context and communication patterns are described in detail in FIG. 13.

Detailed Description of Exemplary Embodiments

FIG. 6 is an illustration of an interaction graph **520**, according to an embodiment of the invention. In this sim-

plified view device nodes are not shown, only party nodes. Each circular node labeled P corresponds to a person, “party” or “contact,” and a registered user may be labeled “U”, and the AI communication agents are labeled CA. Registered user U-602 may use devices (not shown in this figure) for communication. Interaction graph 520 may be stored in memory 512. Interaction graph 520 represents people and software agents as nodes with relationships and relationships or affinities as edges. The dotted edges in the interaction graph 520 may depict communications interaction relationships between registered user U-602, with known party contacts (P-604, P-606, P-608, P-610), and AI communication agents CA-612 and CA-614. In one embodiment, the edges are weighted by the number of historical interactions between parties and where the absence of an edge indicates no previous direct historical interactions. In FIG. 6, node U-602 represents a registered user as a center point or “root” of their interaction network. Surrounding party nodes (P-604, P-606, P-608, and P-610) represent other people connected to the user. These may include friends, family, coworkers, and acquaintances, but may also represent the source of incoming communications from people or spam bots entirely unknown to the user. AI communication agents CA-612 and CA-614 may be actively connected with users and contacts dynamically by the master AI agent 518 when making decisions about incoming communications by considering relationships, common attributes, and history.

In an embodiment, AI communication agents at nodes CA-612 and CA-614 may be configured by the master AI agent 518 to interact with specific parties. In an embodiment, solid lines 616 and 618 in FIG. 6 may represent actual communication session attempts towards the device of user 602 triggered from parties P-606 and P-604 respectively. In this example, attempt 618 represents a repeat connection between parties P-604 and U-602 which already have a strong direct communications interaction relationship, whereas attempt 616 represents an attempt to make an initial connection from a device of party P-606 to a device of party U-602, which has only a single third level human connection via P-608 and P-604.

FIGS. 7A and 7B depict example dashboards 700 of a user application associated with user preferences 524 within communication management server 506 on the user interface, according to an embodiment of the invention. In an embodiment, the dashboards shown in FIGS. 7A and 7B are examples of an application displayed on a smartphone of user devices 502. Dashboard 700 helps users set the preferences for and respond to notifications from the communication management server 506 using communications 702, configurations 704, and notifications 706. Users can manage their incoming communications by using a mobile web application associated with communication management server 506. In an embodiment, notifications 706 may include a control button to approve, reject, or alter suggestions provided by AI communication agents 516.

Communications 702 may provide a list of pending communications 708 and past communications 710 along with master AI agent 518 recommendations and actions 712. In an embodiment, a user may have access to sender profile details 714 including the person’s name, company, location, and photo. In an embodiment, the user can provide feedback 716 on the recommendations and actions from master AI agent 518. In an embodiment, the user can use filter 718 to narrow the view of, e.g., pending communications 708 and past communications 710 by date range, specific senders, or

communication channel. Filters 718 may be used, e.g., for searching for contacts and identifying communication patterns.

Insights 720 provides summary statistics and trends about communications associated with users based on incoming communication of user devices. For example, volume and trends for different channels and sender types may be provided to the user. In another example, insights 720 may be specific to AI communication agents 516 and actions taken. Frequency of handling, redirecting, gathering info, and blocking. Further, in some embodiments, insights 720 provides metrics related to communication. Metrics may include conversational quality and processing of unwanted versus wanted communications. Conversational quality reflects on natural language metrics like fluency, coherence, and accuracy. Unwanted versus wanted communications include a comparison of communications deemed wanted against unwanted.

In an embodiment, goals, and rewards 724 are provided to enable the user to express the overall goals they want to achieve with communication management and to specify rewards that are indicative of a related alignment score. Alignment score may be a quantitative measure of reward captured vs maximum possible. Configurations 704 provides a list of configuration options for setting user preferences 524 and rules for managing communication. Do not disturb (DND) hours 726 may be set by the user. DND may refer to unavailable times to mute notifications. Contact exception 728 may be related to priority contacts. DND may not apply to contact exception 728. Channel Priorities 730 allows users to rank different channels of communication (messaging, email, calls for importance, custom rules) in order of priority. Further, channel priorities may also include rules and conditions to route senders and keywords. AI delegation 732 refers to the level of automation versus user confirmation. In an embodiment, the master AI agent 518 may direct actions without requiring user confirmation via a control button. Activity settings 734 may be related to the visibility of transcripts, recording, and data storage. Integration permissions 736 display the allowed third parties that can access user device data. This type of configuration and communication-related insights may assist in the data-driven tuning of the master AI agent 518 to help in improving alignment and effectiveness.

FIG. 8 illustrates an example of a highly simplified environment state model 800 used by master AI agent 518 for determining an action for incoming communication, according to an embodiment of the invention. In an embodiment, master AI agent 518 invokes ASF 510 to determine the action to be selected for processing the incoming communication. ASF 510 may be associated with the environment state model that implements the MDP to reflect the different states, actions, and associated rewards. In the example shown in FIG. 8, environment state model 800 may be implemented as a fully observable MDP. MDP provides a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision-making agent. MDPs are used to help make decisions that maximize the cumulative reward to the registered user. MDP is a framework for AI model-based reinforcement learning. Environment state model 800 may be configured with states, transitions, and rewards based on the user preferences. The MDP may be characterized by the following:

States (S): A set of states describing the possible combined state of the agent representing the registered user in its environment. In FIG. 8 states labeled SA-804,

SB-806, and SC-808 depict different states corresponding to specific communication and general environment states with a registered user. In an embodiment, a user will have many states for representing variation of communication-related states and many more for additional holistic state combinations. The states in environment state model **800** may be designed based on the user preferences and short-term and long-term goals of the registered user or clustering of GCN embeddings.

Actions (A): A set of actions that can be taken by the master AI agent **518** in any specific state. The black dots on each state transition may reflect the different possible actions taken from that state which will result in a probabilistic transition to the next state indicated by the transition arrow.

Transition Probabilities (P): The transition model is a description of what state the registered user moves to in their environment, given the current state and action taken by the master AI agent **518**. The transition is uncertain and has a probability associated with it for each combination of state and available action. For example, when a spammer is asked a question to discover more information about them or the purpose of the call, they may answer the question or they may abandon the call resulting in transitions to different communication states with different probabilities. In an embodiment, the registered user may start their first day on the system at state “S-802” and may desire to be close to some holistic state “E-810” at the end of the day with messages answered and no communications active and located in their bedroom by 11 pm. Powering on their phone the next morning would result in them being transitioned to a new starting state depending on waiting messages and any missed calls.

Rewards (R): A reward function that returns the immediate reward received after transitioning from one state to another due to an action. Rewards are a measure of fulfillment of user preferences **524** by master AI agent **518**. In an embodiment, different actions may be associated with different rewards

FIG. **9** is a flow diagram of an example method **900** for determining an action for processing an incoming communication using an MDP model-based approach, according to an embodiment of the invention using the MDP model-based approach. The steps of method **900** may be performed at communication management server **506** by the master AI agent **518**.

In the model-based approach, environment state model **522** may be configured to model the state of the user’s world and to predict the outcomes (next states and rewards) of communications actions taken from a particular state the user may be in at the time the action is taken. Because they possess a model of the environment, model-based systems may also plan by considering multiple future states and choose actions that optimize long-term rewards. This capability allows for sophisticated strategies that plan a sequence of future events. A model-based system can therefore use the environment model to simulate and evaluate not only the immediate reward from an individual action but also the accumulated reward of sequences of actions to maximize long-term rewards. In model-based reinforcement learning, the objective is to learn a function that provides a mapping from the current state of the user in their environment to the best action they can take, i.e. the one that maximizes the expected value of the total (discounted) reward in the future. An advantage of the model-based approach is that it can

efficiently make use of prior knowledge and extrapolate to new situations not previously encountered.

At step **902**, master AI agent **518** may receive signaling and routing information related to the incoming communication device via the multimedia gateway **508**. The incoming communication may be directed towards a first-user device registered with the communication management server **506**. The incoming communication may be a voice message, an audio call, a text message, an email, or a video communication.

At step **904**, master AI agent **518** may use a state representation function to determine the current state of the user environment. In an embodiment, a state representation function may update interaction graph **520** based on the incoming communication and may determine the environment state of the user based on a context associated with the incoming communication, user preferences associated with the user device, and the updated interaction graph **520**, and features from the GCN.

In an embodiment, an existing State Representation Function may be used to map the structured and unstructured description into a single state *s* of the set of all states *S* of the model-based Markov Decision Process.

In an embodiment static, dynamic, structured, and unstructured data may be retrieved from interaction graph **520** and user preferences **524** to create a declarative form of the decision problem to be solved. The decision problem may consider structured and unstructured data to generate a current state in the environment state model **522**.

Examples of structured and unstructured data may include Do Not Disturb (DND) hours (set unavailable times to mute notifications), contact exceptions (override hours for priority contacts), channel priorities—rank messaging, email, calls for importance, custom rules (If-then logic to route senders and keywords), AI delegation (desired level of automation versus user confirmation), activity settings (visibility of transcripts, recording, data), integration permissions (allowed 3rd parties and data access), short-term and long-term goals relating to communications decisions that may be found in the user preferences **524** present in memory **512** of communication management server **506**. In addition to user preferences **524**, data from interaction graph **520** including relationship scores with friends, family, and co-workers may also be considered. Besides using data from interaction graph **520** and user preferences **524**, data from users’ calendars, and currently active communications in progress along with their content type and duration may also be incorporated. In an embodiment, the actual data may be a mixture of JSON (JavaScript Object Notation) text for the more structured data and plain text for the unstructured data.

At step **906**, master AI agent **518** determines if the current state of the user environment matches with an existing state. When the current state defined at step **904** matches an existing state in environment state model **522** then at step **908**, the ASF (policy) is called to determine the action to be taken which is returned to the master AI agent **518** at step **916**.

At step **906**, when the current state determined in step **904** does not match with an existing state, then at step **910**, the environment state model **522** is updated with the new state based on the created current state. State representation function may be involved in normalizing numerical values, numerically encoding categorical variables, converting text data into dense vector embeddings, using the embeddings of the GCN, clustering of dense vectors to define states *S*, and validating states with LLM. Environment state model **522** may be updated for the new state using starter data from

25

history if available or using starter data from a Large Language Model (LLM). The updating of environment state model 522 with new states using clustering of dense vector embeddings ensures that state generation does suffer from drawbacks of high dimensionality. Further, in some embodiments, environment state model 522 may be updated to remove old states that are no longer being used.

At step 912 the new MDP may then be solved to find the new ASF.

At step 914 the new ASF is called using the new state to determine the action.

At step 916, the action returned to master AI agent 518 from steps 908 or step 914 is executed. In an embodiment, the action returned by the ASF 510 may be invoked by a function call on the master AI agent 518. For example, the action may be to send an instruction directly to the multimedia gateway 508 to block the incoming call, or the master AI agent 518 function call may require it to invoke one or more generative AI communication agents 516 to perform additional sub-tasks before sending an instruction to multimedia gateway 508.

At step 918, a reward may be generated for the execution of the action. In an embodiment, a reward function may return an immediate reward received after transitioning from one state to another due to an action. Rewards are a measure of fulfillment of user preferences. In an embodiment, rewards may be indicative of a related alignment score. Alignment score may be a quantitative measure of user reward captured vs maximum possible.

ASF 510 may be configured to select an action based on the state of the communications of the registered user in environment state model 522 and the actions with higher reward value may be preferred to optimize long-term rewards. Master AI agent 518 and ASF 510 may use environment state model 522 to simulate and evaluate not only the immediate reward from an individual action but also the accumulated reward of sequences of actions to maximize long-term rewards, noting that the solution of the MDP is the ASF which maximizes the expected reward.

In an embodiment, ASF 510 may be configured to choose actions that maximize the expected value of rewards to the user given the uncertainty of being able to connect and communicate with them, i.e. maximize the probability of connecting multiplied by the reward if connected. In other words, actions may be selected that optimally manage the tradeoff between the dynamic “reachability” of the user to contacts/parties and the reward to the registered user when contact is made. The “reachability” may be the probability of getting to a state of having communicated with a target contact either directly through a single hop or through a more complex path on the interaction graph.

EXAMPLES

When an incoming communication is identified as SPAM, master AI agent 518 may instruct multimedia gateway 508 to block the incoming communication. In an alternate embodiment, master AI agent 518 may instruct multimedia gateway 508 to have the SPAM caller connected to an AI communication agent 516 to answer a SPAM caller. In an embodiment, an AI communication agent 516 may talk to the caller and confirm that the incoming communication is SPAM and then instruct the multimedia gateway 508 to disconnect the call. In another embodiment, an AI communication agent 516 may take the role of “spam-the-spammer timewaster” in which role AI communication agent 516 will first talk with the caller to confirm the call is indeed spam

26

and then deliberately waste the time of the spammer using a convincingly earnest human voice in a pointless, rambling conversation. Contact information of the incoming SPAM communication may be sent to legal authorities and stored in a long-term memory associated with communication management server 506.

When the incoming communication is from a close contact associated with a registered user’s device, an AI communication agent 516 may provide a notification on the registered user’s device. If the registered user accepts a voice call, processing of the incoming communication may be considered complete and a reward may be provided. In some cases, the incoming communication may be redirected to another resource (e.g. another AI communication agent, such as an AI voicemail agent), or the incoming communication may be rescheduled to another time and only a partial reward may be generated.

In case the user decides not to answer a voice call, the call may get routed to a new AI communication agent which may use a friendly and apologetic persona that communicates that the registered user cannot take the call now and will communicate later. In an embodiment, if the calling device is also registered with communication management server 506, AI communication agents associated with both users can negotiate and coordinate an appropriate time to call based on their objectives and constraints and automatically make the connection when the two users are scheduled to be free.

In another example, a registered user of the system, Bob, is working at his company office in San Francisco, his wife is working at another office across town and his son is in high school not far from Bob’s office. Bob has pre-configured his communication management system to assume a persona called “Angel” when communicating with or about his family. The time is 2:35 pm and Bob is in the middle of an important Zoom video conference with a potential business prospect in Dallas. Bob forgot to charge his smartphone overnight and didn’t notice the phone battery was low until midway through the video conference and his smartphone has now died completely and is no longer communicating on any network. Bob makes a mental note to get his charger from his car and charge his phone after his video conference is over. Meanwhile, an urgent security situation has arisen at Bob’s son’s high school. Bob’s wife tried to call Bob 3 minutes ago, but her call went to voice mail because Bob’s phone was dead. Bob’s son tried to send Bob a text message from the school 2 minutes ago. One minute later in the middle of the video conference, Angel proactively and unexpectedly joins Bob’s video conference with the business prospect and says, “Excuse me, and sincere apologies for interrupting this video conference, but there is an urgent situation I need to make Bob aware of” Bob talks to the business prospect who understands and leaves the video conference. Bob tells Angel that his phone is dead and it’s okay to talk about the personal matter on the video conference now. Angel tells Bob about the situation at his son’s high school, that Bob’s wife just tried to call him worried that his son, and that his son had texted him and that she (Angel) in the last 2 minutes has already taken the following actions:

Angel talked to Bob’s wife reassuring her that she is trying to locate Bob and that she is trying to join his video conference to let him know.

Angel replied to the text message from Bob’s son (as Angel) saying that she is locating his father, and he loves him

Angel emailed Bob's manager to say that Bob needs to leave work urgently and requested to reschedule their one-on-one meeting that is currently scheduled for 4 pm today on Bob's calendar

Angel emailed Bob's dry-cleaning company (as Bob) saying he won't be able to pick it up before 5 pm when they close even though it is overdue (based on a reminder Bob had on his calendar)

Still on the video conference with Angel, Bob tells Angel to let his wife know that he's on his way to the school and will charge his phone from his car and asks Angel to put him on a call with his wife as soon as his phone charges enough to rejoin the network and to also try to get his son on a 3-way conference call with his wife while Bob is busy driving. This example highlights the network aspect of the system in the absence of a smartphone and the ability of the system to think and act as a human would in the interests of the human user and understanding a new situation.

FIG. 10 is a flow diagram of an example method for determining an action for processing an incoming communication using model-free and model-based modes, in accordance with an embodiment of the invention.

In model-based reinforcement learning, the decision-making process involves the construction of an internal model of the "world" of the user, also referred to as the "environment." Part of the environment state model covers the communications aspect concerning the state of all communication devices including which devices the user is currently using, who they are communicating with, for how long, on what topic, the number and content of unread messages, and so on. However, a truly "holistic" and "human-centric" view of a user's world that drives their communication behavior needs to also include information not normally considered in telecommunications such as the state of relationships with other people, the current state of mind of the user, current ability to focus, state of tiredness and emotional aspects. Users have long-term goals and also dynamic short-term goals and transient interests, or changes in relationships and responsibilities in their personal and business lives and so on.

Further, modeling a person's world given all possible actions they could perform in that world is a nearly impossible task, however, for now, only the communications actions that can be performed by communication management server 506 are considered. These include simple control actions such as answering, terminating, recording, and redirecting communications, but also more human-like actions such as analyzing the content and tone of a message, inferring the intent, degree of urgency, and so on. Other communication actions include generating content (text, voice, or video) or responding to communications using appropriate emotions and degree of formality. But always doing so using a system and method that ensures that the goals and objectives of the AI in the system are aligned with the goals, objectives, and constraints of the user. Further, accurate modeling of the environment may be complex and computationally expensive.

On the other hand, true "model-free" reinforcement learning may not use an internal model of the user's world environment. Instead, it learns the value of actions directly through trial-and-error interactions with the environment. This approach estimates the rewards associated with different actions and uses these estimates to make decisions. The primary advantage of model-free learning is that it does not require a model of the environment, making it simpler and often more robust to errors in model specification. The downside is that it may require more interactions to learn an

effective action selection function ASF 510, especially in complex environments. These model-free systems cannot generally plan, as they do not anticipate future states beyond learning from direct trial-and-error interactions. Their decision process is short-sighted, focusing on immediate outcomes.

Here by "model-free" it means that there is no explicit internal model of the user environment maintained as was done in FIG. 9 where a state representation function was used to represent the state based on both fully observable state and "latent" state from the GCN and state dynamics under uncertainty described by an MDP. Here in the "model-free" mode, the environment state is only represented "externally" within an LLM by prompting an LLM with the important features from the interaction graph, including fully observable current communication state, relationship states, communication history, and user objectives and preferences in a form of retrieval-augmented-generation relying on the limited reasoning, planning, and prioritization skills of the LLM. In an embodiment, the system may also perform true model-free learning by trial-and-error but generally this results in poor quality of actions early in the learning process which is not acceptable to users, so what is needed is a flexible way to leverage the limited modeling provided by the LLM early in the deployment process in parallel to the model-based approach and dynamically choose the best mode based on performance, history, and novelty of situations being encountered.

In FIG. 10 at step 1002, master AI agent 518 may receive signaling and routing information related to the incoming communication device via the multimedia gateway 508. The incoming communication may be directed towards a first-user device registered with the communication management server 506. The incoming communication may be a voice message, an audio call, a text message, an email, or a video communication.

At step 1004, master AI agent 518 updates the interaction graph.

At step 1006, master AI agent 518 may determine the current performance and cost of available model-free and model-based modes for action determination. In one embodiment, master AI agent 518 may perform decision-making using two different modes of reinforcement learning referred to as "model-based" and "model-free". Master AI agent 518 may switch between these two approaches based on the availability of adequate relevant historical data, the observed decision quality, user goal modifications, or changes in the user's environment. The terms "model-based" and "model-free" are as described above.

At step 1008, ASF 510 may determine if a model-based approach is being used for determining the action. To be more flexible and leverage the advantages of both model-based and model-free learning at different times or in different situations, master AI agent 518 may support both model-based and model-free approaches concurrently, and the system is equipped to automatically adapt to use the best approach.

The relative performance and costs of the model-free and model-based methods for computing the ASF 510 may be evaluated, and one of the following choices may be made; (i) to use only the best-performing mode (ii) to switch to a different mode or (iii) to return an ensembled suggestion (mixture of experts). The method selected may change based on user situation, user settings, available data, and devices of the users. Furthermore, the registered user may begin setting goals that require more complex planning in which case the model-based approach would be required.

At step **1010**, when the model-based method is used by master AI agent **518**, the model-based approach of FIG. **9** is performed to determine the action to be used and the action selected using environment state model **522** may be received for execution by master AI agent **518**.

At step **1012**, a Retrieval Augmented Generation (RAG) style system prompt and user prompt is generated for a generative AI LLM and populated with the important features from the interaction graph, including fully observable current communication state, relationship states, communication history, and user objectives and preferences in a form of retrieval-augmented-generation relying on the limited reasoning, planning, and prioritization skills of the LLM to generate a ranked list of suggested actions.

At step **1014**, to determine action for incoming communication, a generative completion API may be invoked on a foundation model using the prompts generated in step **1012**. In an embodiment, the foundation model may be a pre-trained public foundation model. In another embodiment, the foundation model may be a pre-trained custom foundation model. Generative AI may be a custom model that has been pre-trained to select communication actions to maximize a user reward. The training follows the typical approach of a Chat LLM starting with tasking human labelers to suggest action sequences based on samples presented of holistic states and user objectives, which may be then used to fine-tune a Generative Pretrained Transformer model which then creates candidate actions that are then ranked by human labelers which constitutes a reward model that may be used in a proximal policy optimization loop to train the generative AI for finally generating a ranked sequence of communication actions given user goals and a holistic state description. However, building such a generative model from scratch may be expensive and may lack a large amount of real-world human thinking about life, work, balancing priorities, and general communications. Therefore, a more cost-effective approach for the model may be to simply fine-tune a pre-trained public LLM. In another embodiment, the model-based approach described in FIG. **9** is used to fine-tune the model.

At step **1016**, the foundation model may generate an action list ranked by reward and a highest-ranked action may be selected at step **1018**. At step **1020**, the action is executed.

FIG. **11** is a flow of an example method **1100** for generating relationship fingerprints, according to an embodiment of the invention. Relationship fingerprints may be generated from interaction graph **520** data and communication patterns and may be constantly updated with the processing of new incoming communications. Relationship fingerprints offer relationship characteristics and may be indicative of established communication behaviors.

At step **1102**, pattern-based security manager **532** receives an incoming communication via multimedia gateway **508**. The incoming communication may be directed towards a first-user device among the plurality of user devices **502**. The incoming communication may be a voice message, an audio call, a text message, an email, or a video communication.

In an embodiment, relationship fingerprints may provide the unique characteristics of a communication relationship between two parties and it may include temporal patterns (timing, frequency, duration), channel preferences patterns (voice, text, email), response behaviors (speed, consistency), engagement levels, and relationship context.

The fingerprint continuously evolves as the relationship develops, making it increasingly precise in distinguishing legitimate communications from unwanted ones. Each rela-

tionship generates its own unique fingerprint that adapts over time, enabling sophisticated pattern matching while preserving complete privacy since no message content is ever accessed or stored.

At step **1104**, pattern-based security manager **532** updates interaction graph **520** with context associated with the incoming communication. Interaction graph **520** may maintain nodes representing parties and devices, edges representing relationships, and stores communication history and patterns. Interaction graph **520** provides relationship context. For example, new nodes may be added in case of receiving communication from new user devices, and edge weights may be updated based on interaction frequency from the identified communication patterns.

At step **1106**, pattern-based security manager **532** may use pattern analysis engine **536** to analyze the metadata associated with the incoming communication data to identify patterns. Pattern analysis engine **536** extracts communication patterns present in communication metadata including timing, frequency of communication, channel preferences, and network-level characteristics without accessing communication content.

At step **1108**, pattern-based security manager **532** generates or updates the relationship fingerprint based on the interaction graph **520** data and communication patterns. In case the incoming communication is from a new/unknown user, a new relationship fingerprint may be generated. In case the communication is from a known user and the communication pattern is new, existing relationship fingerprints may be updated with the new pattern. Details related to the update of the relationship fingerprint with the new pattern is described in FIG. **13**.

Each relationship (between two parties) generates its unique fingerprint that adapts over time, enabling sophisticated pattern matching while preserving complete privacy since no message content is ever accessed or stored. Several advantages and features of relationship fingerprints are explained in conjunction with FIGS. **12** and **13**.

FIG. **12** is a flow diagram of an example method **1200** for analyzing the incoming communication metadata to identify patterns.

At step **1202**, pattern-based security manager **532** receives an incoming communication via multimedia gateway **508**. The incoming communication may be directed towards a first-user device among the plurality of user devices **502**. The incoming communication may be a voice message, an audio call, a text message, an email, or a video communication.

At step **1204**, pattern-based security manager **532** updates interaction graph **520** with context associated with the incoming communication. For example, new nodes may be added in case of receiving communication from new user devices. Interaction graph **520** may maintain nodes representing parties and devices, edges representing relationships, and stores communication history and patterns. Interaction graph **520** provides relationship context.

In an embodiment, based on the identified communication patterns interaction graph **520** may be updated. Edge weights may update based on interaction frequency from the identified communication patterns.

At step **1206**, pattern-based security manager **532** may use pattern analysis engine **536** to extract communication metadata associated with the incoming communication data to identify patterns. Pattern analysis engine **536** extracts communication patterns present in communication metadata including timing, frequency of communication, channel

preferences, and network-level characteristics without accessing communication content.

Once the metadata is available, pattern analysis engine 536 may perform different types of analysis simultaneously with the available communication metadata.

In an embodiment, at step 1208, pattern analysis engine 536 may perform real-time communication pattern analysis to handle immediate pattern detection in incoming communications. At step 1214, pattern analysis engine 536 may extract communication patterns related to three main categories: temporal patterns, channel preference patterns, and behavioral patterns. Temporal patterns may be indicative of when and how often people communicate. Temporal patterns may include timing-based patterns like response times, communication frequency, and preferred contact hours. For example, temporal patterns may indicate that a user always responds to work emails within an hour during business hours but takes longer on weekends.

Channel preference patterns highlight preferred communication methods like calls vs. texts, or switching between channels. For example, a user may prefer using text for quick updates and calls for complex discussions. Similarly, channel-switching behaviors (starting with email and then moving to calls for urgent matters) may also be identified.

Behavioral patterns might indicate interaction styles (brief vs. detailed responses), engagement levels (active participation vs. passive responses), and relationship-specific communication habits (formal with clients, casual with teammates).

In an embodiment, at step 1210, pattern analysis engine 536 may perform historical relationship pattern analysis to determine relationship strength, context patterns, and interaction history. Relationship strength may be determined based on the depth and frequency of past interactions.

In an example, relationship strength may be measured through factors like communication consistency (regular weekly meetings vs. sporadic interactions), the longevity of the relationship (years of steady contact vs. recent connections), and interaction depth (detailed collaborative projects vs. surface-level exchanges).

The context patterns may be identified based on the different situations and topics in the incoming communication. In an example, context patterns may include recurring discussion topics (regular financial reviews with clients), situational triggers (emergency response patterns), and role-based interactions (manager-employee one-on-ones).

Interaction history may track how relationships evolve between the parties, such as a customer relationship progressing from initial inquiry to long-term account, including changes in communication frequency, formality levels, and trust indicators.

The real-time communication patterns identified at step 1214 and historical relationship context patterns identified at step 1216 may be used for the generation of a relationship fingerprint between the parties. This relationship fingerprint is constantly updated based on changing patterns, and context between the parties. Further, the relationship fingerprint is also updated with aggregated patterns generated across different services and

In an embodiment, at step 1212, pattern analysis engine 536 may perform cross-pollination analysis, to identify common patterns across services and users/networks. Cross-pollination analysis identifies communication patterns that are found across different services, users, and networks.

At step 1220, pattern analysis engine 536 may identify common patterns that are emerging across different services, users, and networks. In an embodiment, pattern analysis

engine 536 may perform cross-pollination pattern analysis by examining communication behaviors across different network types—cellular, VoIP, messaging platforms, etc. This multi-network view enables the detection of sophisticated patterns that might be invisible when looking at a single network. For example, a legitimate business relationship may show consistent patterns across email, voice, and messaging, while fraudulent communications often show inconsistent patterns across different channels.

Any aggregated pattern identified by cross-pollination analysis is normalized so they can be comparable and validated by pattern analysis engine 536. At step 1222, pattern analysis engine 536 may validate the identified aggregated patterns by checking the authenticity of pattern against known legitimate patterns, absence of conflicting patterns, cross-service pattern matches, and pattern consistency based on expected relationship behavior from relationship fingerprints.

Through cross-pollinated analysis, pattern analysis engine 536 continuously strengthens its pattern recognition capabilities. When similar patterns are observed across different services (email, voice, messaging), the communication management servers 506 understanding of those patterns becomes more refined and accurate. This learning occurs while maintaining strict privacy boundaries and no personal information or content is shared between services.

At step 1222, once the aggregated patterns are validated, then at step 1224 a pattern database 538 is updated. The aggregated patterns stored in pattern database 538 may be used by pattern analysis engine 536 while processing incoming communication. The aggregated patterns may be stored with pattern version, timestamp, source information, and pattern relationship.

Along with using patterns in relationship fingerprints, pattern analysis engine 536 may use aggregated patterns to process incoming communication. Further, in some embodiments, relationship fingerprints may be updated with aggregated patterns.

By analyzing patterns (aggregated patterns) across services and networks, communication management server 506 may be able to identify emerging threat patterns before they become widespread. When unusual patterns are detected in one area, this information is abstracted and shared across the network to enable proactive protection. This creates a self-strengthening security system that becomes more effective as attack patterns evolve, without requiring access to communication content. For example, the same spam pattern may be received by users across voice, SMS, and Email. When a spam pattern is added to communication patterns in relationship fingerprints, any uncommunication communication with this pattern can be identified, even if the incoming communication has a valid relationship context.

FIG. 13 is a flow diagram of an example method 1300 for securely validating incoming communications using pattern-based relationship fingerprints, according to an embodiment of the invention. The steps of method 1300 may be performed at communication management server 506 by pattern-based security manager 532.

At step 1302, pattern-based security manager 532 receives an incoming communication via multimedia gateway 508. The incoming communication may be directed towards a first-user device among the plurality of user devices 502. The incoming communication may be a voice message, an audio call, a text message, an email, or a video communication.

At step 1304, pattern-based decision manager 532 uses a pattern analysis engine 536 to identify current communica-

tion patterns. Current communication patterns may include, but is not limited to, time of communication, channels used, and frequency of the incoming communication.

At step 1306, pattern analysis engine 536 may determine if the relationship between the parties associated with the incoming communication is valid. In an embodiment, pattern-based security manager 532 may consider the relationship history, strength, and context between the user device transmitting the incoming communication and the first-user device (recipient of the incoming communication). The relationship validation may be performed based on the relationship context indicated in interaction graph 520, and relationship fingerprints data.

When pattern analysis engine 536 determines that the relationship between the devices is not valid, then at step 1308, pattern-based security manager 532 may block or redirect the incoming communication by sending an instruction directly to the multimedia gateway 508. In some cases, pattern-based security manager 532 may invoke one or more generative AI communication agents 516 to perform additional sub-tasks before sending an instruction to multimedia gateway 508.

At step 1306, for valid relationships, pattern analysis engine 536, at step 1308, may further check if the current communication patterns identified by pattern analysis engine 536 at step 1304 maps to patterns stored in relationship fingerprints database 534. Patterns in relationship fingerprints database 534 may include, but are not limited to, temporal patterns, channel preference patterns, and behavioral patterns and represent established communication behavior between the two parties.

Relationship fingerprints include both pattern characteristics and strength indicators (described in FIG. 12) that measure the consistency and reliability of observed patterns. As relationship fingerprints are generated through multidimensional analysis that considers historical consistency, cross-channel correlation, and relationship context, patterns require more significant deviations to trigger alerts. This ensures that a comparison of current communication patterns with relationship fingerprints is sufficient for security validation of the incoming communication.

In an example, the parties involved in the communication may be user devices associated with a manager and an employee. Relationship fingerprints may include communication patterns: communications primarily during business hours (9 AM-5 PM), primary channel preference may be Email (80% of communications), secondary channel preference may be video calls for scheduled meetings, regular weekly meeting pattern, and formal communication style. The current communication patterns extracted from metadata of incoming communication may include multiple rapid voice calls late at night.

In an embodiment, in addition to mapping the current communication patterns to patterns stored in relationship fingerprints database 534, the current communication patterns may be mapped to aggregated patterns stored in pattern database 538.

At step 1308, when there is no mapping between the current communication patterns and patterns in relationship fingerprints 534, then at step 1310, pattern analysis engine 536 may determine whether there is valid context for a new pattern. As the relationship between the parties is validated at step 1310 and there is no matching communication pattern in relationship fingerprint database 534, pattern analysis engine 536 may use additional contextual information to determine if the current communication pattern is a new communication pattern between the parties. Additional con-

textual information may include, but is not limited to, current communication context, historical patterns, and cross-service pattern matches.

At step 1310, when there is no valid context for the new pattern, then pattern analysis engine 536 may invoke one or more generative AI communication agents 516 to perform additional sub-tasks before sending an instruction to multimedia gateway 508.

At step 1310, when there is a valid context for the new pattern, then at step 1316, pattern-based security manager 532 may create a new pattern at step 1312, and the new pattern is updated in relationship fingerprint database 534 at step 1314. In some cases, instead of updating with new pattern, existing patterns may be modified based on the incoming communication.

Relationship fingerprint database 534 database architecture supports real-time pattern matching while continuously incorporating new pattern observations, leading to an evolved knowledge base of communication behaviors.

Continuing with the previous example of manager and an employee, there may be changes in the communication pattern when the manager and/or employee works from home, or works from a new location (different time zone), work on a new global project, or have just changed their work schedules. In such a case, a new pattern is created and the existing pattern in relationship fingerprint may be updated.

In an embodiment, pattern-based security manager 532 performs initial security validation before master AI agent 518 determines communication actions.

At step 1316, master AI Agent 518 uses the validation results of communication patterns along with interaction graph 520 to determine an action to be performed to process the incoming communication. Master AI Agent 518 may use the security validation results along with current communication patterns to make appropriate decisions.

Different actions to process the incoming communication may include, but are not limited to, allowing direct communication between parties, redirecting to AI communication agents 516, scheduling for later, and blocking the communication.

At step 1318, a reward is generated based on the alignment of the action with established patterns, relationship dynamics, user preferences 524, and communication goals.

As the secure validation of the incoming communication described methods 1100, 1200, and 1300 work on relationship-based communication patterns making spoofing difficult as attackers may not be able to replicate evolving relationship patterns.

The use of cross-pollination analysis, identification of anomalous patterns, and sharing of the aggregated patterns across services provide proactive protection. Further, the security validation is primarily based on communication metadata analysis and does not require access to communication content.

The skilled person will be aware of a range of possible modifications of the various embodiments described above. Accordingly, the present invention is defined by the claims and their equivalents.

What is claimed is:

1. A system for security validation of incoming communications, the system comprising:
 - a multimedia gateway configured to manage real-time media; and
 - a communication management server comprising one or more processors, a memory, and a plurality of programming instructions stored in the memory, the plurality of

35

programming instructions when executed by the one or more processors causes the one or more processors to: receive an incoming communication directed toward a first-user device among a plurality of user devices, wherein the incoming communication is one of a voice message, an audio call, a text message, an email or a video communication;

analyze, by a pattern analysis engine, metadata associated with the incoming communication to identify communication patterns;

validate, a relationship context between a device associated with the incoming communication and the first-user device based on an interaction graph and relationship fingerprints, wherein the relationship fingerprints are indicative of the communication patterns and the relationship context between the device associated with the incoming communication and the first-user device;

responsive to successful validation of the relationship context, determine if the communication patterns in the incoming communication map to communication patterns present in the relationship fingerprints, wherein the relationship fingerprints are stored in a relationship fingerprints database;

responsive to the mapping between the current communication patterns map to communication patterns present in the relationship fingerprints, provide the relationship context and the current communication patterns to master AI agent for processing of the incoming communication.

2. The system of claim 1, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

responsive to non-mapping between the current communication patterns to the communication patterns present in the relationship fingerprints, determine if there is a valid context for a new pattern, wherein the valid context is determined based on a current communication context, historical patterns, and cross-service pattern matches; and

responsive to a valid context for the new pattern, update the relationship fingerprints with the new pattern.

3. The system of claim 1, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

store temporal patterns in the fingerprints database, wherein temporal patterns are indicative of timing and frequency of communications;

store channel preference patterns in the fingerprints database, wherein the channel preference patterns are indicative of preferred communication methods;

store behavioral patterns in the fingerprints database, wherein the behavioral patterns are indicative of interaction styles and engagement levels;

update the temporal patterns, the channel preference patterns, and the behavioral patterns based on new communications, wherein the pattern analysis engine identifies the temporal patterns, the channel preferences, and the behavioral patterns.

4. The system of claim 1, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

identify, by pattern analysis engine, common patterns across services and networks,

validate, identified common patterns based on authenticity, pattern conflicts, and consistency with relationship fingerprints;

36

responsive to a positive validation, normalize validated patterns; and

store the normalized patterns as aggregated patterns in a pattern database, wherein the relationship fingerprints are updated with aggregated patterns.

5. The system of claim 4, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

share the aggregated patterns across different networks, wherein the aggregated patterns are threat patterns.

6. The system of claim 1, wherein to validate the relationship context, the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

determine relationship strength based on a depth and a frequency of past interactions; and

evaluate communication context based on historical interaction patterns.

7. The system of claim 1, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

continuously update relationship fingerprints database based on new interactions.

8. The system of claim 2, wherein the plurality of programming instructions when executed by the one or more processors causes the one or more processors to:

modify existing communication pattern in the relationship fingerprint based on the new pattern.

9. A computer-implemented method for security validation of incoming communications, the method comprising:

receiving, at a multimedia gateway, an incoming communication directed toward a first-user device among a plurality of user devices, wherein the incoming communication is one of a voice message, an audio call, a text message, an email or a video communication;

analyzing, by a pattern analysis engine, metadata associated with the incoming communication to identify communication patterns;

validating a relationship context between a device associated with the incoming communication and the first-user device based on an interaction graph and relationship fingerprints, wherein the relationship fingerprints are indicative of the communication patterns and the relationship context between the device associated with the incoming communication and the first-user device;

responsive to successful validation of the relationship context, determining if the communication patterns in the incoming communication map to communication patterns present in the relationship fingerprints, wherein the relationship fingerprints are stored in a relationship fingerprints database; and

responsive to the mapping between the current communication patterns map to communication patterns present in the relationship fingerprints, providing the relationship context and the current communication patterns to a master AI agent for processing of the incoming communication.

10. The method of claim 9, further comprising:

responsive to non-mapping between the current communication patterns to the communication patterns present in the relationship fingerprints, determining if there is a valid context for a new pattern, wherein the valid context is determined based on a current communication context, historical patterns, and cross-service pattern matches; and

responsive to a valid context for the new pattern, updating the relationship fingerprints with the new pattern.

37

11. The method of claim 9, further comprising:
 storing temporal patterns in the fingerprints database,
 wherein temporal patterns are indicative of timing and
 frequency of communications;
 storing channel preference patterns in the fingerprints
 database, wherein the channel preference patterns are
 indicative of preferred communication methods;
 storing behavioral patterns in the fingerprints database,
 wherein the behavioral patterns are indicative of inter-
 action styles and engagement levels; and
 updating the temporal patterns, the channel preference
 patterns, and the behavioral patterns based on new
 communications, wherein the pattern analysis engine
 identifies the temporal patterns, the channel prefer-
 ences, and the behavioral patterns.
 12. The method of claim 9, further comprising:
 identifying, by the pattern analysis engine, common pat-
 terns across services and networks;
 validating identified common patterns based on authen-
 ticity, pattern conflicts, and consistency with relation-
 ship fingerprints;
 responsive to a positive validation, normalizing validated
 patterns; and
 storing the normalized patterns as aggregated patterns in
 a pattern database, wherein the relationship fingerprints
 are updated with aggregated patterns.
 13. The method of claim 12, further comprising:
 sharing the aggregated patterns across different networks,
 wherein the aggregated patterns are threat patterns.

38

14. The method of claim 9, wherein validating the rela-
 tionship context comprises:
 determining relationship strength based on a depth and a
 frequency of past interactions; and
 evaluating communication context based on historical
 interaction patterns.
 15. The method of claim 9, further comprising:
 continuously updating the relationship fingerprints data-
 base based on new interactions.
 16. The method of claim 10, further comprising:
 modifying existing communication pattern in the relation-
 ship fingerprint based on the new pattern.
 17. The method of claim 9, further comprising:
 blocking or redirecting the incoming communication
 responsive to unsuccessful validation of the relation-
 ship context.
 18. The method of claim 9, wherein analyzing the meta-
 data associated with the incoming communication com-
 prises:
 performing real-time communication pattern analysis to
 identify temporal patterns, channel preference patterns,
 and behavioral patterns without accessing communica-
 tion content; and
 performing cross-pollination analysis to identify common
 patterns across different communication services used
 by the first-user device.

* * * * *